

# Development of a Web-Based Joint-Level Control Bridge for a Dynamixel-Based Hexapod Robot Toward Raspberry Pi, Inverse Kinematics, ROS, and Reinforcement Learning Integration

---

by

Wong Zhen Hao

Academic Research Paper / Robotics System Development

Institution: Sunway University

Project Type: HUMAC Research Project

Programme: Diploma in Information Technology

Research Area: Robotics, Hexapod Locomotion, Dynamixel Actuator Control, Web-Based Control, Raspberry Pi Deployment, Inverse Kinematics, ROS Integration

Date: May 2026

## Table of Contents

|                                                                     |    |
|---------------------------------------------------------------------|----|
| ABSTRACT.....                                                       | 4  |
| 1. INTRODUCTION.....                                                | 6  |
| 2. PROBLEM STATEMENT.....                                           | 9  |
| 3. LITERATURE REVIEW.....                                           | 10 |
| 3.1 Hexapod Robot Locomotion and Mechanical Design .....            | 10 |
| 3.2 Dynamixel Smart Actuator Control.....                           | 10 |
| 3.3 Inverse Kinematics for Hexapod Robots .....                     | 11 |
| 3.4 ROS and Modular Robotics Software.....                          | 12 |
| 3.5 Research Gap .....                                              | 12 |
| 4. METHODOLOGY.....                                                 | 13 |
| 4.1 Research Design.....                                            | 13 |
| 4.2 Hardware Setup.....                                             | 14 |
| 4.3 Software Architecture.....                                      | 15 |
| 4.4 Dynamixel Motor Mapping and Calibration.....                    | 16 |
| 4.4.1 Repeatable Flat-Pose and Horn Alignment Calibration .....     | 17 |
| 4.5 Movement Control Development .....                              | 21 |
| 4.6 Web-Based, Controller-Based, and Diagnostic Command System..... | 21 |
| 4.7 Safety Monitoring and Testing Procedure.....                    | 23 |
| 4.8 Software Architecture and Project File Organization .....       | 24 |
| 5. RESULTS AND DISCUSSION .....                                     | 27 |
| 5.1 Control Bridge Implementation .....                             | 27 |
| 5.2 Ready-Pose Calibration Results .....                            | 28 |
| 5.3 Raw-Position Conversion and Direction Correction Results .....  | 29 |
| 5.4 Movement Behaviour Results .....                                | 30 |
| 5.5 Web-Based Control Results .....                                 | 31 |
| 5.6 Controller-Client Communication Results .....                   | 33 |
| 5.7 Joint-Limit Probing Results .....                               | 34 |
| 5.8 Diagnostic Status Monitoring Results .....                      | 34 |
| 5.9 Control Bridge Validation Summary .....                         | 35 |
| 5.10 Key Findings.....                                              | 36 |
| 5.11 Practical Engineering Observations.....                        | 37 |

|                                                              |    |
|--------------------------------------------------------------|----|
| 5.12 Communication and Command Flow Lessons.....             | 38 |
| 5.13 Raspberry Pi Onboard Power Integration Result .....     | 39 |
| 6. LIMITATIONS AND FUTURE DEVELOPMENT .....                  | 40 |
| 6.1 <i>Future Work</i> .....                                 | 41 |
| 7. CONCLUSION .....                                          | 43 |
| REFERENCES .....                                             | 44 |
| APPENDIX A: System Architecture and Communication Flow ..... | 45 |
| APPENDIX B: Physical Hardware Setup .....                    | 50 |
| APPENDIX C: Software Package and Controller Scripts.....     | 58 |
| APPENDIX D: Motor Mapping and Ready-Pose Calibration.....    | 67 |
| APPENDIX E: Basic Diagnostic and Health Check Outputs .....  | 71 |
| APPENDIX F: Movement Testing Evidence .....                  | 73 |

## ABSTRACT

This study presents the design, implementation, and experimental development of a web-based joint-level control bridge for a Dynamixel-based hexapod robot. The robot was originally designed around a legacy CM-530 and RoboPlus Motion workflow, where movement is commonly executed using predefined motion pages. Although this approach is suitable for basic demonstrations, it limits flexible integration with modern robotics workflows such as Python-based control, web-based teleoperation, Raspberry Pi deployment, inverse kinematics, custom client-server communication, and reinforcement learning.

The proposed system develops a Python-based hardware and software bridge that maps Dynamixel actuator IDs to physical hexapod joints, stores calibrated ready-pose values, sends controlled raw-position commands, monitors actuator feedback, and provides a web interface for movement control. Instead of depending fully on external robotics middleware at this stage, the project uses custom Python scripts to perform middleware-like functions, including controller translation, client-server communication, command routing, motor-health checking, and movement execution. This provides greater flexibility for local experimentation, especially because the project involves multiple communication layers between controller input, web commands, onboard processing, actuator feedback, and future artificial intelligence or reinforcement learning modules.

The platform uses AX-series Dynamixel actuators, including AX-12A and AX-18A motors, a CM-530 controller and Dynamixel bus structure, serial communication adapters, a custom wiring and distribution board, an onboard Raspberry Pi computing unit, a 3-cell LiPo power source, a regulated 5 V step-down converter for Raspberry Pi power, and external input devices such as a game controller. The Raspberry Pi is integrated into the robot as the onboard Python control server and is powered from the robot's battery system instead of relying on a separate external power source. The system was developed progressively from motor scanning and joint testing into a web-based controller capable of executing forward, backward, strafe, turning, body-height, push-up, and simultaneous gait-style movements.

A major contribution of this work is the documentation of practical hardware discoveries that are often not visible in idealised robotics examples. Testing showed that actuator behaviour depends not only on command values, but also on voltage level, battery condition, motor temperature, mechanical friction, leg balance, load distribution, wiring reliability, and actuator age or condition. The Dynamixel load value was useful as an approximate indicator of motor stress and ground resistance, but it could not be treated as a perfect torque measurement under real hardware conditions. During extended operation, voltage reduction and heat accumulation affected movement consistency, showing that safe locomotion requires both software limits and physical observation.

The project now uses IKControl.py as the main current control script, while SControl.py is preserved as the stable hardcoded joint-space baseline for fallback and comparison. IKControl.py includes fixed gait fallback, IK Motion, IK Bézier Motion, movement mode switching, Web UI control, terminal control, controller-client support, and preliminary Body IK posture testing. This structure allows the robot to remain operational using tested hardcoded movement while also supporting more advanced foot-space and posture-control experiments.

The results show that a practical hexapod control system requires more than simply sending movement commands to motors. A reliable research-ready platform must include motor discovery, joint mapping, ready-pose calibration, movement limits, voltage monitoring, temperature monitoring, load observation, web-based command handling, controller input mapping, stable onboard power distribution, and safe separation between high-level logic and low-level actuator execution. The completed work establishes a foundation for onboard Raspberry Pi operation powered through the robot battery system, custom robotics middleware development, inverse kinematics refinement, reinforcement learning simulation, future ROS-compatible integration, and safer sim-to-real transfer.

**Keywords:** Hexapod robot, Dynamixel, AX-12A, AX-18A, CM-530, Raspberry Pi, web-based robot control, custom robotics middleware, inverse kinematics, joint-level control, hardware bridge, reinforcement learning, actuator diagnostics.

## 1. INTRODUCTION

Hexapod robots are six-legged robotic platforms commonly used to study legged locomotion, stability, terrain adaptation, and robotic control systems. Compared with wheeled robots, hexapod robots can maintain body support while some legs are lifted during movement, making them suitable for experiments involving walking gait design, joint coordination, load distribution, and future artificial-intelligence-based locomotion. However, controlling a hexapod robot is more complex than controlling a simple wheeled platform because each leg normally contains multiple joints that must move in a coordinated and balanced manner. Previous hexapod studies also show that locomotion performance is affected not only by control algorithms, but also by mechanical structure, leg design, and dynamic movement behaviour (Ma et al., 2022).

The hexapod robot used in this project is based on ROBOTIS Dynamixel AX-series smart actuators, including AX-12A and AX-18A motors. These actuators provide position control, actuator feedback readings, and shared bus communication, which allow the robot controller to send movement commands while also reading hardware status values. The ROBOTIS AX-12A documentation explains that actuator data is organised through a control-table structure, where values can be read or modified using instruction packets (ROBOTIS, n.d.). This makes Dynamixel actuators suitable for research-oriented robot control because the software can monitor feedback values such as present position, load, voltage, temperature, and movement status while executing movement commands.

The original robot platform was designed around the CM-530 controller and RoboPlus Motion workflow, where movement is usually created through predefined motion pages. This method is useful for basic demonstrations because the user can create and replay fixed movements. However, it becomes limited when the objective is to develop a more flexible research platform that can support Python-based control, web-based teleoperation, onboard Raspberry Pi operation, inverse kinematics experiments, controller input mapping, and future reinforcement learning integration.

In this project, the focus is not only on making the hexapod walk, but also on developing a practical hardware and software bridge between high-level control logic and low-level Dynamixel actuator execution. The system was designed to map each motor ID to its physical joint location, store calibrated ready-pose values, send controlled raw-position commands, monitor actuator feedback, and provide different control interfaces for testing. These include a terminal interface, a web-based control interface, and a controller-client script that can send movement commands to the server. This approach allows the robot to be tested progressively, starting from individual motor checks and joint calibration before moving toward complete gait execution.

A major motivation for this work is the limitation of idealised robotics examples when applied to real hardware. In theory, a Dynamixel actuator can be controlled by sending a target position value to a motor ID. In practice, the movement result depends on many physical factors, including battery voltage, motor temperature, mechanical friction, joint alignment, wiring reliability, motor age, load distribution, and whether each leg touches the ground evenly. During testing, it was observed that the same command could produce different physical behaviour depending on the robot's balance, battery condition, and accumulated heat. Therefore, this project treats actuator feedback such as position, voltage, temperature, and load as important diagnostic information rather than optional data.

The system went through multiple stages of development. Early work focused on motor scanning, identifying working motor IDs, checking communication reliability, and recovering from hardware inconsistencies after electrical connection issues. Later stages involved capturing a stable ready pose, testing individual joint movement, developing safe movement limits, creating forward and backward gait sequences, adding strafe and turning movements, and improving movement coordination through simultaneous or semi-overlapping gait phases. A web-based controller was then developed to allow easier testing and teleoperation, while a separate client-side controller script allowed external input devices to send movement commands over the local network. The final hardware stage integrated the Raspberry Pi onto the hexapod as the onboard Python control server and powered it from the robot's battery system through a regulated 5 V step-down converter, removing the need for a separate external Raspberry Pi power supply during normal demonstration.

The current software structure separates the stable baseline controller from the main advanced controller. `SControl.py` is preserved as the original hardcoded joint-space controller and remains useful as a fallback and comparison baseline. `IKControl.py` is now treated as the main current controller because it includes the fixed gait fallback, IK Motion, IK Bézier Motion, movement mode switching, terminal control, Web UI control, controller-client support, and preliminary Body IK posture testing. This allows the robot to support both physically tested joint-space movement and more advanced foot-space inverse kinematics movement within the same control framework.

Instead of fully relying on ROS at the current stage, this project uses custom Python scripts that perform middleware-like functions. These scripts handle controller input translation, client-server communication, command routing, motor feedback checking, and actuator command execution. ROS is widely discussed as a robotics software framework that provides a structured communication layer for robot software systems (Quigley et al., 2009). However, the custom Python approach used in this project provides more direct control over communication behaviour during early-stage hardware testing and is easier to modify for the specific needs of the robot.

The main contributions of this study are as follows:

1. Development of a Python-based joint-level control bridge for a Dynamixel-based hexapod robot, reducing dependence on fixed RoboPlus Motion pages and enabling programmable actuator-level control.
2. Implementation of a web-based and terminal-based control system that supports ready-pose execution, forward and backward walking, strafing, turning, body-height adjustment, push-up motion, motor health monitoring, and controller-client communication.

3. Creation of a calibrated motor-to-joint mapping system that links Dynamixel actuator IDs to physical hexapod joints and stores tested ready-pose values for repeatable movement execution.
4. Documentation of real hardware behaviour, including the effects of voltage drop, motor temperature, load variation, wiring reliability, actuator condition, mechanical friction, and leg balance on movement consistency.
5. Development of SControl.py as a stable hardcoded baseline and IKControl.py as the main current controller, supporting fixed gait fallback, IK Motion, IK Bézier Motion, movement mode switching, and preliminary Body IK posture testing.
6. Establishment of a modular control architecture that supports onboard Raspberry Pi operation powered through the robot battery system, custom robotics middleware development, reinforcement learning experiments, and possible ROS-compatible integration.

Overall, this project aims to create a research-ready hexapod control platform that is practical, modular, and extendable. The contribution of the work is not limited to the final walking movement, but also includes the documentation of hardware behaviour, actuator limitations, wiring considerations, safety observations, software architecture, and the development process required to turn a legacy motion-based hexapod into a Python-controlled experimental robotics platform.

## 2. PROBLEM STATEMENT

The original hexapod robot platform was developed around the CM-530 controller and RoboPlus Motion workflow, where movements are commonly created as fixed motion pages and executed through predefined sequences. Although this workflow is suitable for demonstration-based movement, it is not flexible enough for research tasks that require direct joint-level control, real-time command handling, actuator feedback monitoring, web-based teleoperation, controller input mapping, onboard Raspberry Pi operation, inverse kinematics, and future reinforcement learning integration.

A major problem encountered in this project is that real hexapod locomotion is strongly affected by hardware conditions that are not always represented in ideal examples or simulation-based demonstrations. During physical testing, actuator behaviour was influenced by battery voltage, motor temperature, load distribution, mechanical friction, wiring reliability, leg alignment, floor contact, and the condition of older Dynamixel motors. As a result, simply sending target position values to the actuators was not sufficient to guarantee stable or repeatable movement. The robot required careful calibration, ready-pose tuning, health monitoring, and progressive movement testing before reliable movement behaviour could be achieved.

Another challenge is that the robot requires a direct and modular bridge between high-level control commands and low-level Dynamixel actuator execution. Without this bridge, external inputs such as a web button, keyboard command, game controller, inverse kinematics output, ROS node, or reinforcement learning policy would have to interact with the actuator system in an unsafe or unstructured way. A proper bridge is needed to separate user input, command routing, movement generation, motor mapping, calibration values, safety checking, and actuator command transmission.

The project also faces practical hardware reliability issues. During development, motor detection and actuator response were affected by wiring condition, bus topology, power distribution, communication stability, and individual motor behaviour. These issues show that the robot cannot be treated only as a software-control problem. The control system must also support diagnostic feedback, motor status checking, voltage and temperature observation, load monitoring, and repeated hardware validation.

Therefore, the main problem addressed in this study is how to transform a legacy Dynamixel-based hexapod robot from a predefined motion-page platform into a Python-controlled experimental platform. The target system must support calibrated joint-level movement, actuator diagnostics, web-based command handling, controller-client communication, onboard Raspberry Pi control-server operation, safe movement execution, and future extensibility for inverse kinematics, ROS-compatible integration, reinforcement learning, and safer sim-to-real transfer.

### 3. LITERATURE REVIEW

Hexapod robot research commonly focuses on legged locomotion, mechanical stability, gait generation, actuator control, and the relationship between body structure and movement performance. Unlike wheeled robots, hexapod robots require coordinated control across multiple legs and joints. Each movement must consider not only the commanded joint positions, but also the robot's mechanical structure, ground contact, balance, actuator capability, and movement timing. Therefore, a practical hexapod control system requires both software-level movement planning and hardware-level observation.

#### ***3.1 Hexapod Robot Locomotion and Mechanical Design***

Hexapod robots are often studied because six-legged structures can provide stable support during movement. While one group of legs is moving, the remaining legs can support the robot body. However, the performance of a hexapod does not depend only on the gait algorithm. It is also affected by the robot's mechanical design, leg structure, actuator placement, and how movement is transferred from the motor to the foot.

Ma et al. (2022) studied a hexapod robot with a six-link leg mechanism designed for high dynamic locomotion. Their work shows that mechanical structure can directly affect dynamic motion performance, because leg design influences how motor movement becomes foot movement. This supports the idea that real hexapod locomotion cannot be treated as a pure software-control problem. In this project, similar hardware-related effects were observed during physical testing, where leg balance, friction, actuator condition, and body posture affected movement consistency.

#### ***3.2 Dynamixel Smart Actuator Control***

Dynamixel actuators are commonly used in robotics projects because they combine a motor, controller, driver, sensor, reduction gear, and communication capability into one actuator module. For this project, the robot uses ROBOTIS AX-series Dynamixel actuators, including AX-12A and AX-18A motors. These actuators are useful for research because the control system can send position commands while also reading feedback values from the actuator.

The ROBOTIS AX-12A e-Manual explains that Dynamixel actuators use a control table to manage actuator state and control data. Users can read data registers to obtain status information and write data registers to modify actuator behaviour. This supports the use of feedback values such as present position, voltage, temperature, load, and movement status in the control bridge.

For this project, actuator feedback is important because the hexapod cannot be controlled safely by sending movement commands alone. During testing, voltage drop, temperature rise, load variation, and inconsistent motor response affected the robot's movement behaviour. Therefore, the control bridge treats Dynamixel feedback values as diagnostic information for hardware observation and safety checking.

### 3.3 Inverse Kinematics for Hexapod Robots

Inverse kinematics is an important method in legged robot control because it allows the desired position of a robot foot to be converted into joint angles. Instead of manually commanding each joint with raw position offsets, an inverse kinematics system can calculate the required coxa, femur, and tibia joint angles based on the target foot position. This is useful for future movement control because the robot can be commanded using leg or foot targets instead of only individual motor values.

Lizarraga et al. (2024) discuss inverse kinematics and omnidirectional movement for hexapod robots, showing that kinematic methods can support more precise and adaptable leg movement. This is related to the current direction of this project, where IKControl.py is used as the main advanced controller for foot-space movement. The implementation uses measured coxa, femur, and tibia segment lengths to convert target foot displacement into hip, femur, and tibia joint commands.

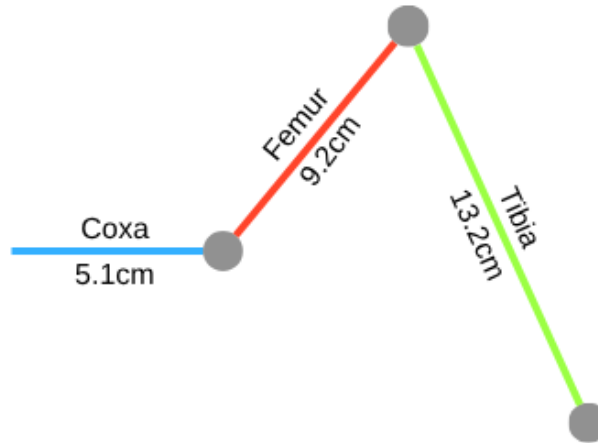


Figure 3.1: Simplified Coxa, Femur, and Tibia Segment Lengths Used for Hexapod Leg Kinematics

However, inverse kinematics alone does not solve all real hardware problems. Even if the calculated joint angles are mathematically correct, the physical robot still needs calibration, direction correction, safe joint limits, actuator feedback monitoring, and mechanical validation. For this reason, IKControl.py is treated as the main current controller, while SControl.py remains preserved as a stable hardcoded fallback and comparison baseline. Although the IK system has been implemented, further calibration is still required to match the mathematical foot targets with exact physical movement on the real robot.

### ***3.4 ROS and Modular Robotics Software***

Modern robotics systems often use middleware to separate different parts of the robot software system. For example, one module may handle sensor input, another module may handle movement planning, another may handle actuator control, and another may handle user commands. This separation makes the system easier to expand and maintain.

Quigley et al. (2009) describe ROS as a robotics framework that provides a structured communication layer above the host operating system. ROS is not an operating system in the traditional sense, but it supports communication between robot software components. This idea is relevant to the current project because the Python control bridge performs middleware-like functions even though full ROS integration has not yet been implemented.

In this project, the custom Python system handles command routing, controller input translation, web communication, movement execution, motor mapping, actuator feedback reading, and safety checking. This custom approach is useful during early-stage hardware testing because it allows direct control over the robot’s communication and movement behaviour. Future ROS-compatible integration can later build on top of the tested Python bridge instead of replacing it completely.

### ***3.5 Research Gap***

Existing robotics studies and documentation provide important foundations for hexapod locomotion, actuator control, inverse kinematics, and modular robotics software. However, many examples focus on ideal movement models, clean hardware conditions, simulation, or completed robot designs. In practical development, especially with reused or older hardware, the robot may behave differently from theoretical expectations.

The gap addressed by this project is the development and documentation of a practical hardware-control bridge for a real Dynamixel-based hexapod robot. Instead of focusing only on final autonomous walking, this project focuses on the foundation required before advanced control methods can be safely applied. This includes motor discovery, motor-to-joint mapping, ready-pose calibration, raw-position command conversion, web-based control, controller-client communication, voltage and temperature monitoring, load observation, joint-limit probing, and hardware debugging.

This research gap is important because future systems such as inverse kinematics, ROS nodes, reinforcement learning policies, and simulation-to-real transfer require a reliable low-level bridge before they can safely control physical motors. Without this bridge, high-level commands may produce unsafe actuator movement, excessive load, unstable posture, or hardware damage. Therefore, the contribution of this study is not only the movement behaviour of the robot, but also the structured development of a safer and more extendable joint-level control platform.

## 4. METHODOLOGY

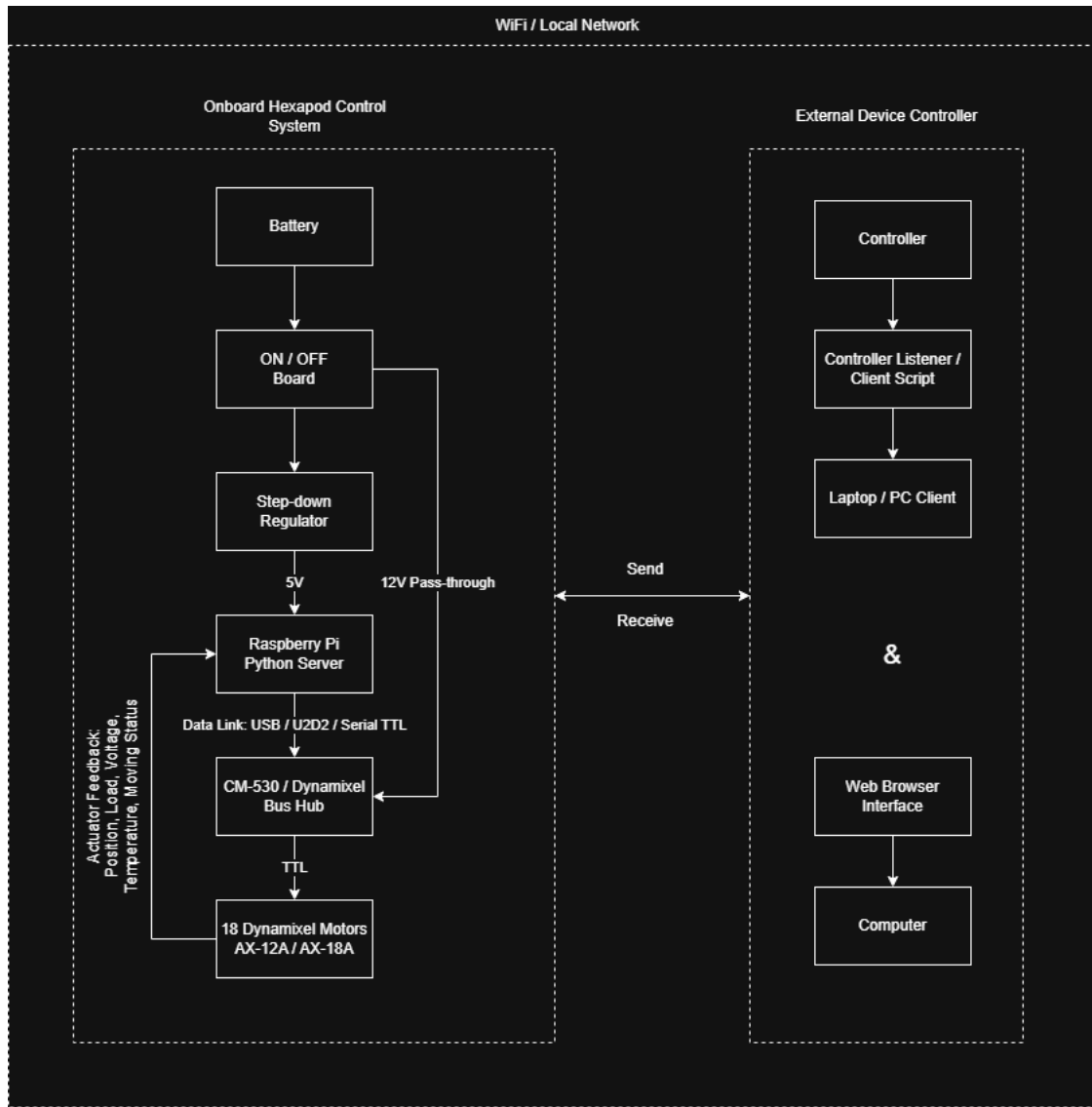
This study uses an experimental development approach to design, implement, and test a Python-based joint-level control bridge for a Dynamixel-based hexapod robot. The methodology focuses on converting a legacy motion-page-based hexapod system into a more flexible control platform that supports direct actuator command, web-based teleoperation, controller input handling, actuator feedback monitoring, and future expansion toward inverse kinematics and reinforcement learning. The work was carried out progressively through hardware inspection, motor discovery, joint mapping, ready-pose calibration, movement testing, web interface development, controller-client communication, and safety observation.

The methodology is divided into several stages. The first stage involved identifying the robot hardware structure, including the Dynamixel actuator arrangement, CM-530 controller connection, TTL daisy-chain wiring, power supply, and communication adapters. The second stage involved scanning the Dynamixel bus, identifying active motor IDs, mapping each motor to its physical joint, and recording actuator feedback. The third stage focused on building a stable ready pose and developing basic joint-level movements. The fourth stage involved creating walking, strafing, turning, body-height, and push-up behaviours. The final stage involved integrating the movement system with a web interface and controller-client communication while observing voltage, temperature, and load feedback during operation. A final hardware integration stage was then added to mount the Raspberry Pi onto the hexapod and power it from the onboard battery system through a regulated 5 V step-down converter, allowing the web-control server to operate on the robot without a separate external Raspberry Pi power source.

### *4.1 Research Design*

The research design follows a practical hardware-in-the-loop development process. Instead of relying only on simulation or predefined motion files, each major function was tested directly on the physical hexapod robot. This was necessary because the robot's behaviour was affected by real hardware conditions such as actuator age, mechanical friction, wiring reliability, leg balance, voltage drop, motor temperature, and load distribution. Therefore, the system was developed through repeated testing, observation, adjustment, and validation.

The development process was not limited to writing movement code. It also included identifying communication problems, checking motor responses, recording working poses, testing safe movement limits, and comparing different control strategies. This approach allowed the system to evolve from simple motor scanning into a working web-based control bridge capable of controlling the robot through different command interfaces.



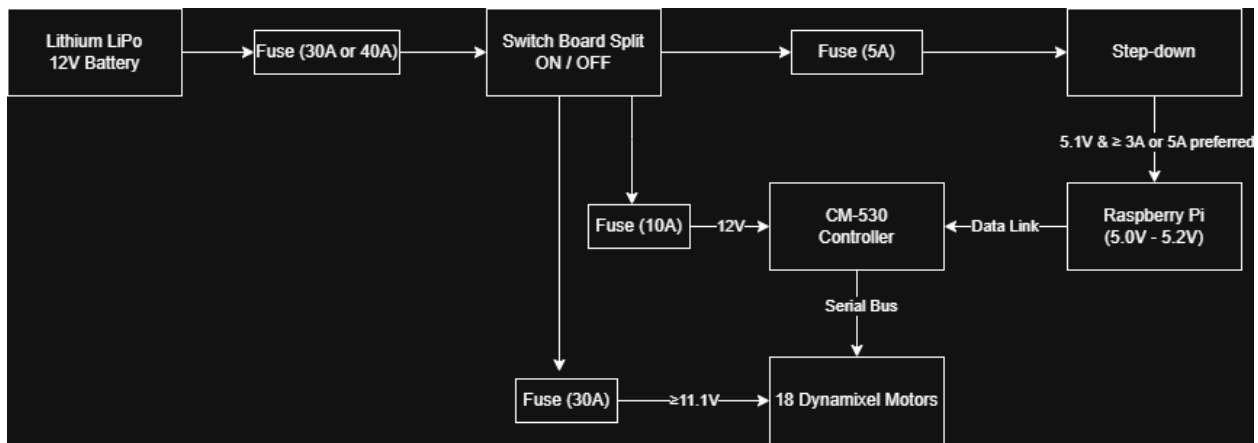
**Figure 4.1: Final System Architecture Map of the Web-Based Joint-Level Hexapod Control Bridge**

#### 4.2 Hardware Setup

The hexapod robot uses 18 ROBOTIS Dynamixel AX-series smart actuators arranged across six legs. Each leg consists of three joints: hip or coxa, femur, and tibia. The robot uses AX-12A and AX-18A actuators, with the stronger actuator type used where higher joint load is expected. The actuators communicate through a TTL daisy-chain bus, where multiple motors share the same communication line.

The robot is powered using a 3-cell LiPo battery, with the motor system supplied through the main power path and the Raspberry Pi supplied through a regulated 5V step-down converter. The Dynamixel actuators receive power through the robot's motor power distribution path, while the Raspberry Pi requires a lower operating voltage. Therefore, a step-down voltage regulator is used to reduce the battery voltage to a safe voltage level for onboard computing. The CM-530 controller is retained as part of the existing robot wiring and bus structure, while the U2D2 USB-to-Dynamixel TTL adapter is used to transmit control packets from the Python control system to the Dynamixel bus.

The final hardware and communication flow is organised around the onboard Raspberry Pi Python server, the CM-530/Dynamixel bus structure, and the external wireless control device. The main robot battery supplies the Dynamixel actuator system through the existing motor power distribution path, while a separate step-down voltage regulator reduces the battery voltage to a stable 5 V supply for the Raspberry Pi. This allows the Raspberry Pi to operate from the onboard robot battery instead of relying on an external power source. The Raspberry Pi is mounted onto the robot platform and operates as the onboard Python control server. It communicates with the Dynamixel control path through the available USB, U2D2, or serial TTL data link. External devices such as a laptop, web browser, or controller-client script can connect through the same Wi-Fi or local network and send high-level movement commands to the Raspberry Pi server. The Raspberry Pi then routes these commands through the IKControl.py bridge before movement commands are sent to the Dynamixel actuators.



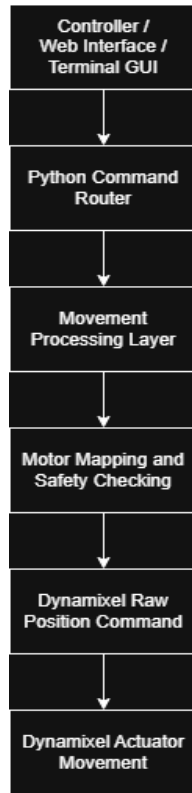
**Figure 4.2: Onboard Raspberry Pi Power Distribution and Communication Flow Between the Battery, Step-Down Regulator, CM-530, and Dynamixel Actuators**

### 4.3 Software Architecture

The software architecture is built around custom Python scripts that perform middleware-like functions. Instead of depending fully on ROS at the current stage, the project uses custom scripts to handle command routing, controller input translation, web communication, movement generation, actuator feedback reading, and safety checking. This provides direct control over the communication structure and allows the system to be modified quickly during hardware testing.

The main current controller is named IKControl.py. This script acts as the primary server and movement controller for the latest system. It provides terminal control, web-based control, controller-client communication, fixed gait fallback, IK Motion, IK Bézier Motion, movement mode switching, preliminary Body IK Posture testing, motor health checking, and actuator feedback monitoring. The SControl.py script is preserved as the stable original hardcoded joint-space controller for fallback and baseline comparison. ControllerClient.py is used as the current controller/gamepad client for sending high-level movement commands to the server.

The software command flow begins when a user sends a high-level command through the terminal, web interface, or controller-client script. The Python server receives the command, checks the command type, routes it to the appropriate movement or diagnostic function, applies the stored motor mapping and ready-pose calibration values, and then sends the resulting raw-position targets to the Dynamixel actuator bus. This structure separates user input from low-level motor execution and allows movement behaviour, safety logic, and diagnostic monitoring to remain inside the main control bridge.



**Figure 4.3: Software Command Flow from Web Interface and Controller Client to Dynamixel Actuator Execution**

#### ***4.4 Dynamixel Motor Mapping and Calibration***

A key stage of the methodology was mapping each Dynamixel motor ID to its physical joint on the hexapod. Since the robot has 18 actuators, accurate motor-to-joint mapping is required before reliable movement can be produced. Each motor ID was associated with a specific leg and joint type, such as front-left hip, front-left femur, front-left tibia, and so on.

After motor identification, a stable ready pose was manually tuned and recorded. This ready pose acts as the calibrated base position for all movement commands. The control system uses this base pose to calculate target positions by applying degree-based offsets to the corresponding Dynamixel raw values. This prevents the movement system from relying only on arbitrary raw positions and allows movement to be expressed more clearly as a combination of calibration data and logical joint offsets.

The formula used in the control system can be described as:

$$\text{raw\_target} = \text{base\_raw} + \text{logical\_deg} \times \text{RAW\_PER\_DEG} \times \text{leg\_movement\_sign} \times \text{joint\_direction}$$

where `base_raw` is the calibrated ready-pose value, `logical_deg` is the intended joint movement in degrees, `RAW_PER_DEG` converts degrees into Dynamixel raw position units, and the sign values correct the movement direction for each joint and leg.

#### ***4.4.1 Repeatable Flat-Pose and Horn Alignment Calibration***

In addition to mapping each Dynamixel motor ID to a physical joint, a repeatable horn-alignment calibration method was developed so that the control code can be reused on a similarly assembled hexapod robot. This is important because two robots may use the same Dynamixel IDs and the same Python control script, but their physical starting pose may still be different if the servo horns are installed at different angles.

The calibration method separates the idea of a code-perfect pose from a physical-flat pose. The code-perfect pose uses clean common raw values as the software reference. In this project, the reference flat pose was based on approximately 520 for the coxa joints, 505 for the femur joints, and 365 for the normal tibia joints. For the two mechanically inverted tibia joints, the reference value is approximately 665 instead. These values provide a repeatable software midpoint for horn installation and make the robot easier to align before movement testing.



**Figure 4.4: Star-Shaped Flat-Pose Setup Used for Servo Horn Alignment**

Figure 4.4 shows the hexapod laid flat in a star-shaped posture during the horn-alignment calibration process. This view is useful for checking the overall leg symmetry, especially the coxa alignment around the robot body.

The first calibration stage is performed with the robot laid flat in a star-shaped posture. Each leg is extended outward so that the coxa, femur, and tibia joints can be visually aligned. The motors are commanded to the code-perfect reference values, and the servo horns are installed so that the physical leg alignment matches this reference posture as closely as possible.

The coxa joints are mainly aligned visually because their purpose is to set the direction of each leg around the body. This can be checked using Dynamixel Wizard 2.0 or by commanding the motors near their midpoint position and confirming that the legs are physically symmetrical. The femur and tibia joints require more detailed calibration because small horn offsets and mechanical differences can cause the feet to touch the ground unevenly.



**Figure 4.5: Oblique View of the Flat-Pose Horn Alignment Setup**

Figure 4.5 shows the same flat-pose calibration setup from an oblique viewing angle. This view helps show the femur and tibia alignment more clearly, while Figure 4.4 shows the overall leg symmetry and coxa alignment from above.

After the horns are installed, the robot is placed into its supported standing posture and the legs are pushed down until the feet contact the ground evenly. The CM-530 and Dynamixel system are then powered again, and the position/status script is used to read the actual present position of each motor. These readings represent the physical-flat pose of the assembled robot. The physical-flat pose may differ slightly from the code-perfect pose because of horn teeth spacing, bracket tolerance, actuator orientation, leg weight, and mechanical load.

The difference between the code-perfect reference values and the measured physical-flat values is treated as the mechanical offset of the real hardware. This offset is not considered a software error. Instead, it records how the real assembled robot differs from the ideal reference pose.

Table 4.1: Repeatable Flat-Pose Calibration Procedure

| Step | Action                                                     | Purpose                                                                             |
|------|------------------------------------------------------------|-------------------------------------------------------------------------------------|
| 1    | Lay the hexapod flat in a star-shaped posture              | Allows all six legs to be visually compared                                         |
| 2    | Command the motors to the code-perfect reference pose      | Creates a common software alignment reference                                       |
| 3    | Align and install the servo horns                          | Matches the physical horn position to the software reference as closely as possible |
| 4    | Check the coxa joints visually                             | Ensures each leg points outward in the expected direction                           |
| 5    | Place the robot into its supported standing posture        | Prepares the robot for physical-flat reading                                        |
| 6    | Push the legs down until the feet contact the floor evenly | Reveals real mechanical offsets caused by horn spacing and bracket tolerance        |
| 7    | Run the position or status reading script                  | Captures the actual present position of each motor                                  |
| 8    | Update the ready-pose dictionary if needed                 | Makes the control code match the assembled robot more closely                       |

This calibration method allows another user to assemble a similar hexapod, align the horns using the same flat-pose reference, record their own physical-flat motor readings, and update the ready-pose dictionary. As a result, the same control code can produce a more similar starting posture even when the exact servo horn installation and mechanical tolerances are not perfectly identical.

#### ***4.5 Movement Control Development***

Movement control was developed progressively. Initial testing began with single-joint movement and individual leg lifting to observe safe motion ranges and actuator response. After the ready pose was established, basic movement patterns were created for forward and backward walking. These were later expanded to include strafing, turning, body-height adjustment, push-up movement, and gait-style locomotion.

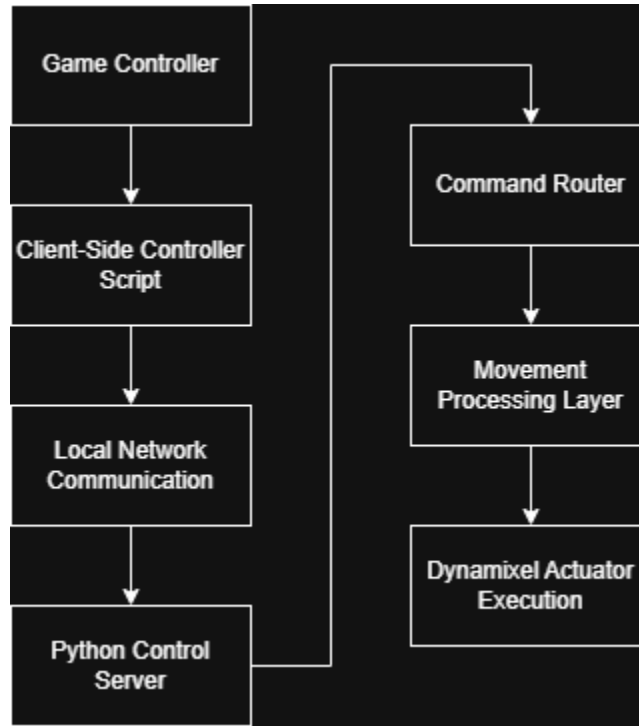
The stable movement branch uses calibrated joint-space control. In this approach, each movement is generated from tested joint offsets rather than full mathematical foot-position targeting. This method was selected as the stable baseline because it was physically tested and adjusted based on real robot behaviour. The controller uses tripod-style gait logic, where alternating groups of legs move while the remaining legs support the body.

The latest movement system is implemented in IKControl.py. This controller supports multiple movement profiles, including Fixed Gait, IK Motion, IK Bézier Motion, Bézier Demo Motion, and Body IK Posture testing. In IK Motion, leg movement is generated from foot-space targets and converted into hip, femur, and tibia commands using the measured coxa, femur, and tibia lengths. IK Bézier Motion extends this by applying a smoother curved trajectory during the swing phase. Body IK Posture testing adjusts body height, pitch, and roll by modifying all six leg targets through the IK layer. Although these features are now implemented and tested, further calibration is still required for accurate physical lift height, body geometry, and long-duration movement validation.

#### ***4.6 Web-Based, Controller-Based, and Diagnostic Command System***

A web-based interface was developed to make the hexapod easier to test, control, and demonstrate. The web interface allows the user to send movement commands such as ready pose, forward walking, backward walking, strafing, turning, stopping, body-height adjustment, push-up motion, and health checking. This reduces the need to manually type repeated commands through the terminal and provides a more practical control method during physical testing.

In addition to the web interface, a controller-client script was developed to allow an external game controller to send commands to the robot server. The game controller is connected to the client device, while the final onboard configuration uses the Raspberry Pi as the robot-side Python server. The Raspberry Pi is mounted on the hexapod and powered from the onboard battery system through a regulated 5 V step-down converter. When the Raspberry Pi and external client device are connected to the same local Wi-Fi network, the controller-client script sends high-level movement commands to the Raspberry Pi server through local communication. The server then translates these commands into movement routines and Dynamixel actuator position commands.



**Figure 4.6: Controller-Based Communication Flow from Game Controller to Dynamixel Actuator Execution**

This custom communication approach was selected because the project requires direct control over many communication layers, including web commands, controller input, actuator feedback, movement execution, and future artificial intelligence or reinforcement learning modules. Instead of depending fully on an external robotics framework at this stage, the custom Python-based communication system provides a more flexible and modifiable structure for local experimentation.

A separate set of diagnostic and calibration scripts was also included to support repeatable hardware setup. The status snapshot script prints the current state of all 18 Dynamixel actuators, including present raw position, degree offset from the ready pose, load value, voltage, temperature, moving state, and warning condition. This information was used to observe actuator condition, identify unstable joints, and capture the current physical standing pose of the robot as a ready-pose dictionary.

In addition to status printing, automatic joint testing and probing scripts were prepared to estimate the practical physical movement limits of the robot. The mechanical retry limit probing script moves each selected joint gradually in the positive direction until the motor can no longer follow the target position. When this happens, the script retries the same target several times before recording the last successful position as the practical positive limit. The motor is then returned to the ready position and tested in the negative direction using the same process. This allows the system to record approximate minimum and maximum usable joint ranges for each actuator.

These diagnostic tools were important because the physical range of a joint is not always the same as the theoretical range of the Dynamixel actuator. Mechanical assembly, horn alignment, friction, wiring condition, actuator age, load distribution, and robot posture can all reduce the usable movement range.

Therefore, the calibration and probing scripts were used to separate theoretical actuator capability from the practical joint limits that are safer for real gait development.

This confirms that the web and controller-based command system can operate through the onboard Raspberry Pi server without requiring the Raspberry Pi to be powered by a separate external supply during normal demonstration.

#### ***4.7 Safety Monitoring and Testing Procedure***

Safety monitoring was included throughout the development process because the hexapod's movement behaviour changed depending on physical hardware conditions. During testing, voltage, temperature, and load readings were observed to detect possible stress conditions. Voltage reduction affected movement consistency, rising motor temperature indicated actuator strain, and high load readings suggested resistance, imbalance, or possible mechanical obstruction.

The Dynamixel load value was treated as an approximate diagnostic indicator rather than an exact torque measurement. This is because the reading can be affected by mechanical friction, battery voltage, joint angle, actuator age, ground contact, and the internal condition of the motor. Therefore, load readings were used together with visual observation, movement response, and temperature readings when evaluating whether a movement was safe.

The testing procedure followed a gradual hardware-in-the-loop process. First, the communication port and baud rate were selected. The Dynamixel bus was then checked to confirm that the motors responded correctly. After this, a status snapshot was taken to observe present position, voltage, temperature, and load. The robot was moved to the ready pose before any movement command was tested. Simple joint movements were tested before full gait movements, and health checks were repeated between tests to monitor actuator condition.

The general testing procedure is summarised as follows:

1. Connect the robot power supply and communication adapter.
2. Open the selected communication port.
3. Scan or check the Dynamixel actuator responses.
4. Print the motor status snapshot.
5. Capture or confirm the ready-pose values.
6. Test individual joint movement.
7. Probe practical joint limits if needed.
8. Execute simple body or leg movement.
9. Test walking, strafing, and turning behaviours.
10. Observe voltage, temperature, load, and physical movement.
11. Adjust movement parameters and repeat testing.

This safety-first procedure helped reduce the risk of unstable movement, excessive actuator load, overheating, and unexpected mechanical collision. It also made the development process more repeatable because each major movement change could be tested together with actuator feedback and physical observation.

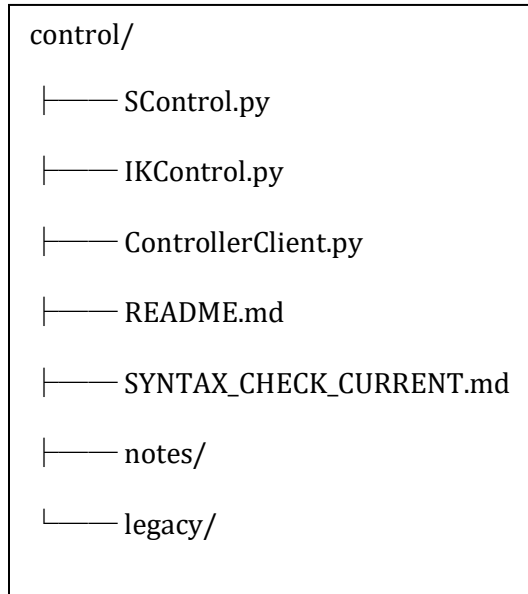
## 4.8 Software Architecture and Project File Organization

The hexapod control software was reorganized into a cleaner project structure to separate the current working system from experimental versions, legacy files, hardware assets, documentation, and supporting tools. This organization makes the project easier to maintain, test, explain, and extend in future development stages.

The cleaned project folder is divided into several main directories:

```
Hexapod_Research_Public_Clean/  
|  
├── control/  
├── experiments/  
├── legacy/  
├── hardware/  
├── docs/  
├── tools/  
├── model/  
├── README.md  
└── PROJECT_STRUCTURE.md
```

The control/ folder contains the main working control scripts used for the current robot system. This folder is the most important directory for daily development and testing. It includes the stable hardcoded controller, the latest inverse kinematics controller, and the controller client script.



The file `SControl.py` is the stable original controller based on the hardcoded joint-space gait. This version is preserved as a reliable fallback and baseline comparison. It uses predefined joint movement values for the hip, femur, and tibia joints.

The file `IKControl.py` is the main current control script. It includes fixed gait fallback, inverse kinematics movement, Bézier foot trajectory testing, movement mode switching, Web UI control, terminal control, controller-client support, and preliminary Body IK Posture control.

The file `ControllerClient.py` is the current controller/gamepad client script. It is used to send controller input to the main control system, allowing movement commands such as forward, backward, strafing, turning, ready pose, reset, and height-related controls.

The `control/legacy/` folder contains older control versions that are still useful for reference but are no longer the main working files. For example, the older web-only version is preserved as a legacy reference.

The `experiments/` folder stores selected experimental milestones from the development process. These include older inverse kinematics test versions, gait tuning attempts, strafe experiments, turning experiments, and other movement development branches. These files are not the main files to run daily, but they provide an important record of how the control system evolved.

The `legacy/` folder contains older preserved baselines and previous project versions. These are kept for historical reference and fallback comparison. They are not used as the main current control system.

The `hardware/` folder contains mechanical and hardware-related assets. This includes CAD/STL files for robot body and leg components, as well as hardware-related reference files. The STL files are useful for mechanical documentation, future 3D printing, and body geometry reference when improving body IK.

The docs/ folder contains the project documentation files, including the research report and written notes that are directly relevant to the working project. This folder is used to separate documentation from executable code.

The tools/ folder contains calibration and diagnostic helper scripts. These tools are used for tasks such as checking motor status, probing joint limits, printing ready poses, and supporting hardware debugging. They are not the main walking controller, but they help validate and maintain the robot system.

The model/ folder contains kinematics and robot model reference files. These files are useful for representing the robot's physical structure and can support further development of inverse kinematics, body IK, and future reinforcement learning experiments.

This file organization improves the maintainability of the project by clearly separating stable working code, current research code, experimental versions, hardware assets, and documentation. It also supports safer development because new movement experiments can be tested without overwriting the stable baseline controller.

| Folder / File               | Purpose                                                                                                                    |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------|
| control/SControl.py         | Stable hardcoded gait controller and fallback baseline                                                                     |
| control/IKControl.py        | Main current controller with fixed gait fallback, IK Motion, IK Bézier Motion, mode switching, and Body IK Posture testing |
| control/ControllerClient.py | Current controller/gamepad client                                                                                          |
| control/legacy/             | Older control scripts kept for reference                                                                                   |
| experiments/                | Selected experimental movement and IK development versions                                                                 |
| legacy/                     | Older preserved project baselines                                                                                          |
| hardware/                   | CAD/STL files and hardware assets                                                                                          |
| docs/                       | Documentation and report files                                                                                             |
| tools/                      | Calibration and diagnostic helper scripts                                                                                  |
| model/                      | Kinematics and robot model reference files                                                                                 |

## 5. RESULTS AND DISCUSSION

This section presents the implementation results of the proposed joint-level hardware control bridge and the experimental tests conducted on the Dynamixel-based hexapod robot. The objective of this section is to evaluate whether the developed Python-based bridge can detect motors, read actuator feedback, map motor IDs to physical joints, record calibrated ready-pose values, send controlled movement commands, and support safety-limited joint testing.

Unlike a pure simulation study, this work involves physical robot hardware. Therefore, the results include both successful software implementation outcomes and practical hardware observations discovered during testing. These observations are important because real robot platforms often produce issues that are not visible in theoretical models, such as inconsistent motor detection, wiring dependency, actuator movement constraints, and serial communication instability.

### 5.1 Control Bridge Implementation

The current main control system is implemented in IKControl.py. This script acts as the primary Python server and movement controller for the latest hexapod system. It provides terminal-based control, web-based control, controller-client communication, fixed gait fallback, IK Motion, IK Bézier Motion, movement mode switching, preliminary Body IK Posture testing, movement execution, motor health checking, and actuator feedback monitoring. The older SControl.py controller is preserved as a stable hardcoded joint-space baseline for fallback and comparison.

The control bridge separates high-level user commands from low-level actuator execution. A user may send a command through the terminal, web interface, or external controller-client script. The Python server receives the command, routes it to the correct movement function, applies motor mapping and calibration values, and sends raw Dynamixel position targets to the actuator bus.

Table 5.1: Dynamixel Motor ID to Hexapod Joint Mapping

| Motor ID and Joint Name |                  |
|-------------------------|------------------|
| ID 1 = RL_hip           | ID 10 = ML_femur |
| ID 2 = FL_hip           | ID 11 = MR_tibia |
| ID 3 = FR_femur         | ID 12 = ML_tibia |
| ID 4 = FL_femur         | ID 13 = RR_hip   |
| ID 5 = FR_tibia         | ID 14 = FR_hip   |
| ID 6 = FL_tibia         | ID 15 = RR_femur |
| ID 7 = MR_hip           | ID 16 = RL_femur |
| ID 8 = ML_hip           | ID 17 = RR_tibia |
| ID 9 = MR_femur         | ID 18 = RL_tibia |

This mapping became one of the most important foundations of the project because all movement commands depend on correct motor identification. If a motor ID is mapped incorrectly, a command intended for one joint may move another joint instead, which can cause unstable posture, mechanical strain,

or unexpected leg movement. The mapping also allowed the software to display meaningful joint names instead of only numerical motor IDs, making debugging easier during testing.

Motor discovery was also used as a hardware debugging technique. When a motor did not respond, the system could identify whether the issue was likely related to communication, wiring, power, or the actuator itself. This was especially important because the robot had previously experienced electrical connection issues, making consistent motor detection necessary before movement testing.

## ***5.2 Ready-Pose Calibration Results***

The ready-pose calibration process was updated after identifying that the earlier calibration approach was too dependent on the robot's previous physically imbalanced posture and mixed inverted-joint behaviour. In the earlier stage, the ready pose was mainly captured from the robot's physical standing position. Although this allowed movement testing to continue, it did not clearly separate the ideal software reference pose from the real mechanical offsets caused by servo horn placement, joint orientation, and assembly tolerance.

The improved calibration process uses a more repeatable reference-based method. Instead of treating every physically captured value as the starting reference, the robot is first aligned using clean average calibration values. The coxa or hip joints use approximately 520 as the common reference value, while the femur joints use approximately 505. The normal tibia physical-flat reference is approximately 365. These values act as the code-perfect calibration reference and make it easier to compare each joint against a common baseline.

After the robot was placed into the flat-pose calibration setup, the physical-flat readings were recorded and compared with the clean reference values. This produced a clearer mechanical offset record for each joint. Most tibia joints were close to the 365 reference, with only small differences, while the femur joints showed larger mechanical variation caused by horn alignment, bracket position, and real assembly differences. The MR hip remained a mechanical outlier and used approximately 575 instead of the common 520 hip reference.

The captured physical-flat calibration values were stored separately in the CALIBRATION\_PHYSICAL\_FLAT\_RAW dictionary. This dictionary records the measured real hardware pose and is not treated as the final movement-ready standing pose. Instead, it is used to identify how far each real joint differs from the clean reference values. This separation makes the calibration process easier to understand because the project now distinguishes between the code-perfect reference, the physical-flat measured pose, and the final calibrated READY\_POSE.

The final READY\_POSE dictionary was then updated as the default base posture used by the control system. This ready pose acts as the initialized standing and movement reference for later commands. Movement calculations use this calibrated base position and apply logical joint offsets from it, instead of starting from arbitrary raw motor values. As a result, the robot can produce more consistent movement behaviour because the control script begins from a known hardware-specific posture.

This updated calibration method improves repeatability compared with the previous approach because another user can first align their robot to the same clean reference values, record their own physical-flat readings, identify their mechanical offsets, and then update their own READY\_POSE dictionary. The result

is not a universal ready pose that fits every hexapod perfectly, but a more systematic calibration workflow that makes the same code easier to reproduce on similar hardware.

The final calibration evidence is provided in Appendix D. The appendix includes the flat-pose setup images, motor ID mapping, the captured physical-flat calibration dictionary, the final READY\_POSE dictionary, and the motor status output used to verify the calibrated state.

### **5.3 Raw-Position Conversion and Direction Correction Results**

The stable controller uses joint-level raw-position control based on the calibrated Dynamixel motor positions. After the calibration process, the final READY\_POSE dictionary is treated as the base posture for movement execution. The separate CALIBRATION\_PHYSICAL\_FLAT\_RAW dictionary is used to understand the real mechanical offsets of the robot, while the READY\_POSE dictionary is used as the practical movement reference during operation.

The controller converts intended logical movement in degrees into Dynamixel raw-position offsets. The general formula used by the controller is:

$$\text{raw\_target} = \text{base\_raw} + \text{logical\_deg} \times \text{RAW\_PER\_DEG} \times \text{leg\_movement\_sign} \times \text{joint\_direction}$$

In this formula, base\_raw is the calibrated value from the final READY\_POSE dictionary, logical\_deg is the desired movement offset in degrees, RAW\_PER\_DEG converts degrees into Dynamixel raw-position units, and the sign values correct the movement direction for different legs and joints.

This formula-based structure improved the readability and maintainability of the controller because movement could be described using logical degree values instead of only raw Dynamixel numbers. It also helped separate calibration data from movement behaviour. The calibration dictionaries store the physical hardware reference, while the movement offsets describe how far each joint should move from that calibrated reference.

Testing showed that raw-position conversion alone was not enough to guarantee correct physical movement. Some joints required different direction signs because of mirrored leg placement, actuator orientation, servo horn installation, and mechanically inverted joints. Therefore, the controller includes joint-direction correction and leg-movement sign logic so that a logical movement command produces the intended physical result on each side of the robot.

This direction-correction layer was especially important for making the same movement command behave consistently across left and right legs. Without this layer, a positive movement offset could cause one joint to move in the correct direction while another mirrored joint moved in the opposite physical direction. By combining the calibrated READY\_POSE, logical degree offsets, and direction-correction signs, the controller can generate repeatable raw-position targets while still keeping the movement logic readable.

### 5.4 Movement Behaviour Results

The current IKControl.py controller supports multiple movement behaviours while preserving the earlier hardcoded gait as a fallback mode. After the updated ready-pose calibration, these movements are executed from a more consistent calibrated base posture rather than from the earlier physically imbalanced standing pose. The physically tested movement functions include ready pose, forward walking, backward walking, strafing, turning, body-height adjustment, push-up motion, stop commands, IK Motion, IK Bézier Motion, Bézier Demo Motion, and Body IK Posture testing. These functions allow the robot to be tested from both terminal and web-based interfaces.

Table 5.2: Movement Behaviours Supported by the Controller

| <b>Movement</b>              | <b>Actions</b>                                                                       |
|------------------------------|--------------------------------------------------------------------------------------|
| Ready pose                   | Moves all motors to the calibrated standing position.                                |
| Forward movement             | Uses tripod-style gait logic to move the robot forward.                              |
| Backward movement            | Uses reversed gait direction based on the same calibrated movement structure.        |
| Strafe left and strafe right | Moves the robot sideways by coordinating leg direction and support movement.         |
| Turn left and turn right     | Rotates the robot in place using different leg movement directions.                  |
| Body-height adjustment       | Changes the standing height by adjusting femur and tibia positions.                  |
| Push-up motion               | Moves the body up and down for actuator testing, posture testing, and demonstration. |
| Stop command                 | Stops continuous movement and returns the system to a controlled state.              |
| Calibration/ready update     | Uses the calibrated READY_POSE as the base posture before movement testing           |

During testing, the stable controller became the main operational baseline because it was tuned directly using physical robot behaviour. After the new calibration method was applied, the movement result became easier to interpret because the robot started from a cleaner and more repeatable base posture. This reduced the effect of the earlier imbalanced calibration state and made later gait tuning more consistent across legs, except for known mechanical outliers.

The walking movement was improved progressively by adjusting lift height, support push, gait timing, and movement coordination. Earlier movement attempts showed that if the legs did not lift high enough, the feet could drag against the ground and push the robot backward or sideways. Increasing the lift height improved clearance, but excessive lift increased actuator load. Therefore, the final movement settings were the result of balancing clearance, stability, load, motor temperature, and real foot contact.

### 5.5 Web-Based Control Results

The web-based control interface was successfully developed to make the robot easier to operate, test, and demonstrate. Through the web interface, the user can send movement commands without repeatedly typing terminal commands. The interface supports common robot commands such as ready pose, forward movement, backward movement, strafing, turning, movement stop, body-height control, push-up motion, and health checking. This made physical testing more convenient because repeated movement and diagnostic commands could be triggered directly through browser buttons.

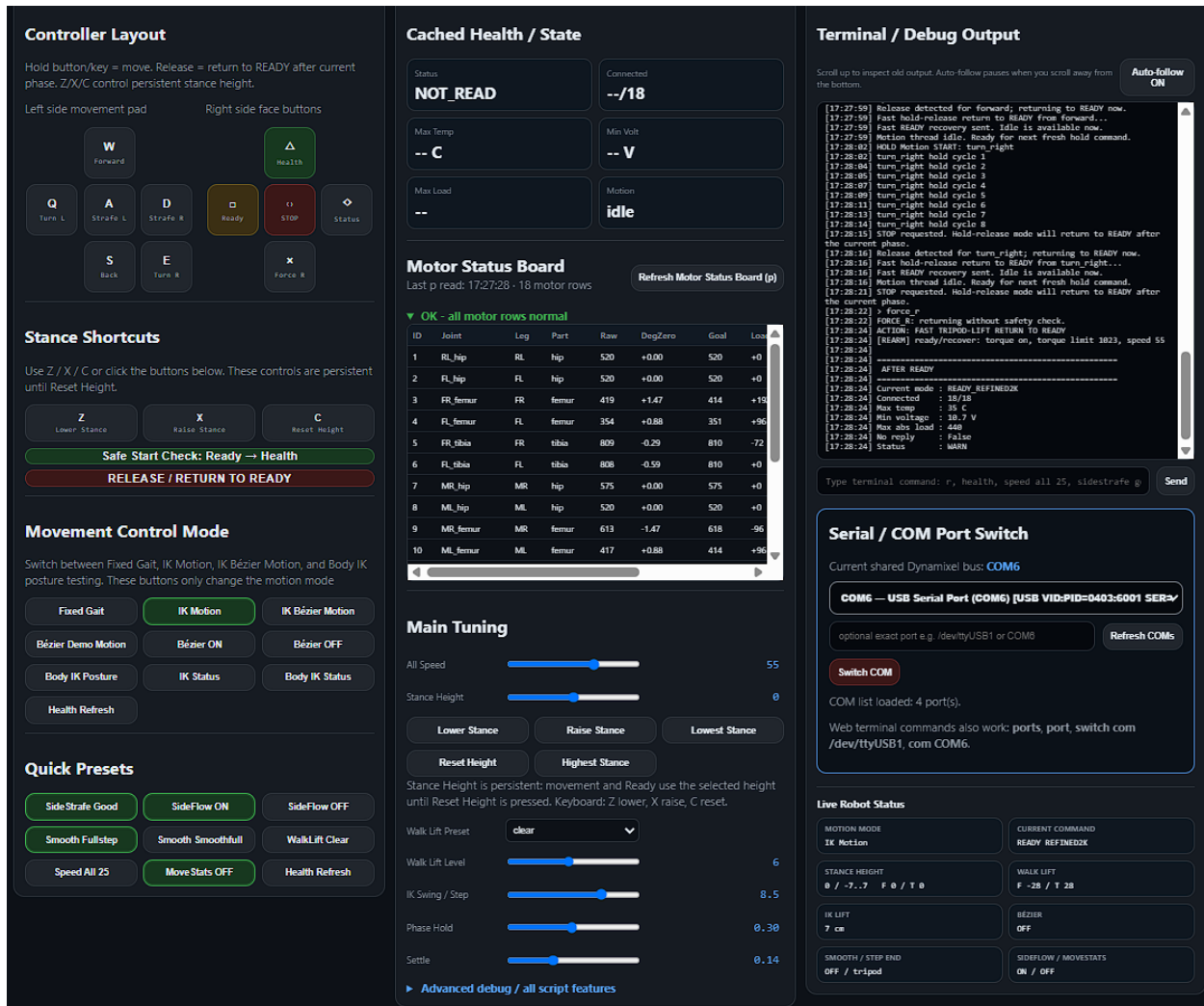
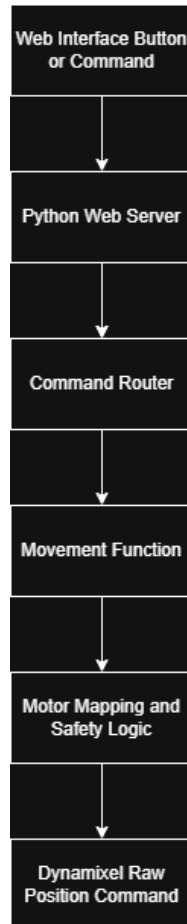


Figure 5.1: Web-Based Hexapod Control Interface Showing Movement, Mode, Tuning, and Diagnostic Panels

Figure 5.1 shows the updated web-based control interface used during physical testing. The interface provides access to movement commands, movement mode selection, tuning controls, health checking, terminal output, and motor-status feedback. This makes the interface useful not only for demonstration, but also for calibration, movement adjustment, and diagnostic observation during hardware testing.

The web-based system also introduced a command routing layer between the browser interface and the actuator control logic. User input from the browser is received by the Python server, processed by the command router, and then passed to the movement processing layer. The movement layer applies the stored motor mapping, calibrated READY\_POSE, movement mode selection, direction correction, and safety-related checks before sending the final raw-position targets to the Dynamixel actuators.



**Figure 5.2: Web-Based Command Flow from Interface Button to Dynamixel Raw Position Command**

Figure 5.2 shows the command flow from a web-interface button press to the final Dynamixel actuator execution. This structure is important because the web interface does not directly send arbitrary raw motor values. Instead, the Python bridge controls how each command is interpreted, routed, converted, and executed. This separation makes the system safer and easier to expand later.

During development, the web layer also revealed software integration issues. Some commands worked correctly in terminal mode but initially failed in web mode because the web command router did not support the same command set. This led to improvements in the routing logic so that terminal commands, fixed-gait commands, IK commands, diagnostic commands, and movement-mode commands could be handled more consistently.

A useful improvement in the web interface was the addition of clearer status and diagnostic display areas. Terminal output alone was not always convenient because new logs could push older diagnostic information out of view. By displaying recent motor-status snapshots and warning information inside the web interface, the user could review actuator position, load, voltage, temperature, moving state, and warning conditions more easily during testing.

The web interface therefore became more than a simple control panel. It acted as a networked testing layer for verifying whether the movement system could be accessed through browser-based commands, which is important for onboard Raspberry Pi operation, local Wi-Fi control, and future controller-client or robotics middleware integration.

## 5.6 Controller-Client Communication Results

A controller-client script was developed to allow an external game controller to operate the robot through local network communication. In the final onboard configuration, the game controller is connected to a client device such as a laptop, while the main Python server runs on the Raspberry Pi mounted on the hexapod. Both devices connect through the same local Wi-Fi network, allowing the controller-client script to send high-level movement commands to the robot-side server.

This structure means that the controller does not directly control the Dynamixel motors. Instead, the controller sends command requests to the Python server, and the server decides how to translate those commands into actuator movement. This separation is important because low-level motor execution remains inside the main control bridge, where motor mapping, calibrated `READY_POSE` values, movement mode selection, direction correction, and safety-related checks are applied before commands are sent to the actuators.

Testing showed that the controller-client structure made the robot easier to operate during demonstrations because movement commands could be sent through a familiar physical controller instead of typing commands manually. This was especially useful for repeated forward, backward, strafing, turning, stopping, and posture-related commands during physical testing.

The controller-client system also helped validate the modularity of the software architecture. The same movement functions could be triggered from different input sources, including terminal commands, web-interface buttons, and external controller input. This shows that the control system is not tied to a single user interface. Instead, different input methods can send high-level commands into the same Python command-routing and movement-execution layer.

During testing, the controller-client setup also confirmed the importance of stable local-network communication. Since the client device and Raspberry Pi communicate over Wi-Fi, connection stability, correct server IP address, port configuration, and firewall or network access must be correct before reliable controller operation can occur. This makes the controller-client system useful for practical teleoperation, but it also shows that network setup becomes part of the robot testing procedure.

Overall, the controller-client result confirms that the robot can be operated through an external input device while still preserving the safety and structure of the Python control bridge. This provides a practical foundation for later integration with ROS-compatible nodes, AI-based decision systems, or reinforcement learning policies, where external modules can send high-level commands without directly bypassing the calibrated low-level actuator control layer.

### ***5.7 Joint-Limit Probing Results***

Joint-limit probing was performed to estimate the practical movement range of each joint. This is different from the theoretical servo range. Although AX-series Dynamixel actuators support a wide raw-position range, the robot's actual mechanical structure may not allow every joint to safely move across that full range.

After the updated ready-pose calibration, joint-limit probing became more meaningful because each joint could be tested from a known calibrated base posture. The final `READY_POSE` provides the starting reference, while probing determines how far each joint can safely move away from that reference before reaching mechanical resistance, body collision, awkward posture, cable interference, or excessive strain.

During probing, selected joints were moved gradually in small steps while their physical motion was observed. The goal was not to force the motor to its maximum possible range, but to identify safe movement boundaries before the joint reached unsafe mechanical conditions. This is important because a value that is technically valid for a Dynamixel servo may still be unsafe for the assembled robot leg.

This process showed that physical joint limits must be discovered experimentally. Bracket design, horn alignment, cable placement, actuator orientation, body geometry, and load distribution can all reduce the usable range of a joint. Therefore, the safety-limit layer should eventually store different minimum and maximum values for each joint rather than applying one universal range to all motors.

The probing process also showed that some joints require additional inspection before being used in walking gaits. If a joint cannot retract, extend, or return smoothly, it should not be included in full-body walking tests until the cause is understood. This is especially important for a hexapod because one abnormal joint can affect the stability of the entire robot.

Overall, the joint-limit probing stage helped convert the project from basic actuator control into a safer robotics control framework. The results support the decision to place a safety-limited bridge between any future high-level controller and the physical motors.

### ***5.8 Diagnostic Status Monitoring Results***

The diagnostic scripts and web-based status panel were able to read useful feedback values from detected Dynamixel actuators, including present position, voltage, temperature, load, moving state, and warning condition. These values are important because a robot control system should not only send movement commands, but also observe the condition of the hardware before and after movement.

Present position feedback allowed the system to confirm whether each motor was physically located near its expected calibrated `READY_POSE` value. This was useful during ready-pose verification, physical-flat calibration checking, and movement testing. Voltage readings helped check whether the actuator bus was receiving sufficient power, while temperature readings were useful for identifying overheating risk during repeated tests. Load readings provided a rough indication of mechanical resistance, uneven ground contact, actuator strain, or possible obstruction.

The diagnostic status output was useful during calibration because it allowed the present position of each actuator to be recorded after the robot was physically adjusted. This made it possible to compare clean reference values, measured physical-flat values, and final ready-pose values. As a result, the diagnostic layer supported both hardware monitoring and calibration documentation.

A useful improvement was the addition of clearer status display inside the web interface. Terminal output alone was not always convenient because repeated logs could push older diagnostic information out of view. By recording the latest motor-status snapshot in a web-based status panel, the system made it easier to inspect actuator condition during testing. This was especially useful when checking whether a specific motor showed abnormal load, temperature, voltage, movement state, or warning status.

This feedback layer is especially important for future development. A walking gait, inverse kinematics module, or reinforcement learning controller may generate many commands quickly. Without status monitoring, the system may continue sending movement commands even when a motor is overloaded, disconnected, overheating, stalled, or not responding correctly. By including diagnostic feedback in the bridge, the robot can later support safer decision-making before movement commands are executed.

The diagnostic results also helped identify that some hardware problems could not be solved by software alone. If a motor is missing from the scan, returning incorrect packets, showing abnormal status values, or behaving inconsistently under load, the issue may be caused by wiring, power, motor configuration, horn alignment, mechanical friction, or physical damage. This confirms the importance of keeping diagnostic tools as part of the research package rather than treating them as temporary test scripts.

### ***5.9 Control Bridge Validation Summary***

The completed control bridge was validated by confirming that high-level commands could be routed safely through the software layers before reaching the Dynamixel actuators. Commands from the terminal, web interface, and controller-client script were able to trigger movement and diagnostic functions through the same Python-based control structure. This shows that the robot is not limited to one input method and that different control interfaces can share the same calibrated movement layer.

The validation process also confirmed the importance of calibration before movement execution. The system now separates the clean code-perfect reference pose, the measured physical-flat calibration readings, and the final calibrated `READY_POSE`. This separation allowed the robot to start from a more consistent hardware-specific posture before movement commands were tested. As a result, later movement behaviour could be evaluated more fairly because it was based on a known calibrated starting condition rather than an older imbalanced physical pose.

The bridge was also validated through raw-position conversion and direction correction. Logical movement commands were converted into Dynamixel raw targets using the calibrated `READY_POSE` as the base. Direction signs were then used to correct mirrored joints, reversed actuator orientations, and mechanically different leg arrangements. This allowed the same logical movement command to produce more consistent physical movement across different legs.

The diagnostic layer further supported validation by allowing actuator position, voltage, temperature, load, moving state, and warning condition to be checked during testing. This feedback confirmed that the system could observe hardware state instead of blindly sending commands. The motor-status output and web-based status display were useful for identifying abnormal actuator behaviour, verifying calibration values, and monitoring hardware stress during repeated movement tests.

Overall, the control bridge was validated as a practical low-level foundation for future robotics development. It supports calibrated joint-level movement, multiple command interfaces, diagnostic feedback, onboard Raspberry Pi operation, and modular expansion toward inverse kinematics, ROS-compatible communication, reinforcement learning, and safer simulation-to-real transfer. The validation result does not mean that all movement behaviours are fully optimized, but it confirms that the platform now has the required control structure for further development.

### ***5.10 Key Findings***

Several key findings were observed during the development and testing of the hexapod control bridge.

First, reliable hexapod movement depends on accurate motor mapping and calibration before any gait or inverse kinematics behaviour is tested. If a Dynamixel ID is mapped incorrectly, a movement command may affect the wrong physical joint and cause unstable posture or unsafe movement.

Second, the updated calibration process showed that the robot should not rely only on a previously captured physical standing pose. A clearer method is to separate the clean code-perfect reference values, the measured physical-flat readings, and the final calibrated `READY_POSE`. This made the calibration process easier to explain, easier to repeat, and more consistent across the legs, except for known mechanical outliers.

Third, the final `READY_POSE` should be treated as a hardware-specific calibration record rather than a universal theoretical pose. Even if another robot uses the same code and motor IDs, small differences in servo horn placement, bracket alignment, actuator orientation, and mechanical load can produce different physical results. Therefore, another user should repeat the flat-pose calibration procedure before expecting the same movement behaviour.

Fourth, actuator feedback is essential for practical robot testing. Present position, voltage, temperature, load, moving state, and warning status helped identify whether the robot was operating near the expected calibrated pose and whether any motor showed abnormal behaviour during movement.

Fifth, the web interface and controller-client system improved usability, but they also showed the importance of command routing. Terminal commands, web buttons, controller input, fixed-gait commands, IK commands, and diagnostic commands must all pass through a consistent Python bridge instead of directly controlling raw motor values.

Finally, physical testing showed that movement performance is affected by both software and hardware conditions. Gait timing, lift height, body posture, voltage level, motor temperature, foot contact, mechanical friction, and joint offsets all influenced the final movement result. This confirms that the control bridge is necessary as a safe foundation before adding more advanced systems such as ROS integration, reinforcement learning, or autonomous movement planning.

### 5.11 Practical Engineering Observations

Several practical engineering observations were discovered during repeated hardware testing. These observations became an important part of the project because they showed that a real hexapod robot cannot be evaluated only by whether the software command is mathematically correct. The same command may produce different physical results depending on actuator condition, voltage level, stance balance, mechanical friction, floor contact, calibration method, and whether the motors have enough time to reach their target positions.

First, voltage level affected movement consistency. When the supply voltage dropped, the same gait command could produce weaker movement, slower response, or less complete leg motion. This showed that voltage monitoring is not only a safety feature, but also a movement-quality indicator. A gait that appears stable at one battery level may become unstable when the battery voltage is lower.

Second, Dynamixel load readings were useful for identifying possible stress, imbalance, ground resistance, or poor stance alignment. However, the load value could not be treated as an exact torque measurement. During testing, load readings were affected by foot contact, leg angle, body posture, mechanical friction, actuator age, and whether the leg was supporting the body evenly. Therefore, load readings were most useful when interpreted together with physical observation, voltage readings, temperature readings, and movement response.

Third, stance tuning was essential for stable movement. Small changes in the ready pose, tibia angle, femur position, or body height affected how evenly the robot stood on all six legs. If one leg touched the ground too strongly or another leg barely supported the body, the robot could become unbalanced even before walking started. This made ready-pose calibration one of the most important foundations of the project.

Fourth, the updated calibration process showed that directly copying a physical standing pose is not always the best calibration method. If the captured pose already contains imbalance, inverted-joint effects, or uneven foot pressure, those issues can become part of the stored ready pose. The improved method separates clean code-perfect reference values, measured physical-flat readings, and the final calibrated READY\_POSE. This made the calibration process more repeatable and easier to explain, while still allowing hardware-specific offsets and mechanical outliers to be recorded.

Fifth, walking performance depended on both software timing and physical motor response. Some movement sequences appeared correct in code, but the robot did not complete the intended physical motion because the motors did not have enough time to reach the target positions before the next phase began. This led to repeated tuning of lift height, support push, phase timing, settling delay, and simultaneous movement behaviour.

Sixth, smoother movement was not automatically better. Bézier foot trajectory testing helped reduce sharp movement and created a smoother swing path, but larger lift arcs or more extreme trajectory settings increased actuator load and could contribute to voltage sag. This showed that smoothness, speed, stability, and motor load must be balanced rather than maximised independently.

Finally, the communication structure confirmed the importance of keeping a safety-limited control bridge between user input and the motors. Web buttons, keyboard commands, controller-client signals, IK outputs, and future reinforcement learning actions should not directly control raw motor values. They should pass through the Python control bridge so that motor mapping, ready-pose calibration, movement limits, safety checks, and diagnostic monitoring remain active before commands reach the Dynamixel actuators.

## 5.12 Communication and Command Flow Lessons

The project clarified how communication should be organised between the user interface, Python control server, controller-client input, onboard Raspberry Pi, CM-530/Dynamixel bus structure, and smart actuators. The web interface, keyboard shortcuts, terminal commands, and game controller do not directly move the motors. Instead, they send high-level command requests to the Python server.

The server then identifies the command, selects the correct movement or diagnostic function, applies the motor ID mapping, uses the calibrated `READY_POSE` as the movement base, checks the selected movement mode, applies direction correction, and sends the final raw-position targets to the Dynamixel actuator bus. This structure prevents external input sources from directly controlling arbitrary raw motor values.

This communication structure is important because the robot now uses multiple command sources. A movement command may come from the terminal, a web-interface button, a controller-client script, or a future ROS, AI, or reinforcement learning module. Regardless of where the command comes from, it should pass through the same Python bridge so that calibration values, movement limits, safety checks, and diagnostic monitoring remain active.

The onboard Raspberry Pi setup also confirmed the importance of separating client-side input from robot-side execution. In the final configuration, the Raspberry Pi runs the robot-side Python server on the hexapod, while external devices such as a laptop, browser, or controller-client script send commands through the same local Wi-Fi network. This makes the robot easier to operate wirelessly while still keeping low-level actuator execution inside the calibrated control bridge.

This structure is also useful because Dynamixel motors are smart actuators that receive packet-based commands and provide feedback values through their control table. The control bridge can send target position commands while also reading present position, voltage, temperature, load, moving state, and warning condition. This makes the system different from a simple motor driver setup because each actuator can provide diagnostic information back to the controller.

During testing, this layered communication structure made debugging easier. If a movement failed, the problem could be separated into different layers: user input, web command routing, controller-client communication, Python movement logic, movement mode selection, motor ID mapping, ready-pose calibration, serial communication, bus wiring, power supply, or physical actuator behaviour. This helped

prevent every problem from being treated as a coding error and made it easier to identify whether an issue was caused by software, wiring, power, calibration, or mechanical resistance.

Overall, the communication-flow result confirms that the project successfully developed a modular command bridge rather than a single-purpose control script. This bridge is important for future work because ROS-compatible nodes, AI-based decision systems, or reinforcement learning policies can later send high-level commands into the same routing layer without bypassing the safety-limited actuator-control structure.

### 5.13 Raspberry Pi Onboard Power Integration Result

The Raspberry Pi onboard integration was completed by connecting the Raspberry Pi to the hexapod's main battery system through a regulated step-down voltage converter. Previously, the Raspberry Pi control server could be tested using an external power source, but this still required the robot to depend on extra wiring during operation. The updated hardware arrangement allows the Raspberry Pi to receive a regulated 5 V supply from the onboard battery system, while the Dynamixel actuator system remains supplied through the main motor power path.

This result confirms that the Raspberry Pi can function as the onboard control host instead of acting only as an external development device. In the final setup, the Raspberry Pi is mounted on the robot and runs the Python-based web-control server. External devices such as a laptop, web browser, or controller-client script can connect through the same local Wi-Fi network and send high-level movement commands to the Raspberry Pi server. The server then routes these commands through the IKControl.py bridge before sending movement commands to the Dynamixel actuator system.

The onboard power arrangement improves the practicality of the hexapod platform because the robot no longer requires a separate Raspberry Pi power supply during demonstration. This makes the system closer to a self-contained web-controlled hexapod platform. However, the Raspberry Pi power branch must remain separated from the motor power rail through the step-down converter because the Raspberry Pi requires a stable 5 V supply, while the Dynamixel actuator system operates from the higher battery voltage. This separation reduces the risk of voltage mismatch and supports safer onboard operation.

The integration also confirmed the importance of stable power distribution in a physical robotics platform. The Dynamixel motors can create high current demand during walking, turning, body-height adjustment, and push-up movements. If the motor load increases suddenly, the battery voltage and wiring stability can affect both actuator behaviour and onboard computing stability. Therefore, future long-duration testing should continue to observe battery voltage, Raspberry Pi stability, motor load, cable movement, and possible undervoltage behaviour during repeated movement tests.

Overall, the onboard Raspberry Pi power result shows that the robot is no longer only a desktop-connected test platform. It can now operate as a more practical onboard web-control system, where the computing unit, command server, battery supply, and actuator-control bridge are integrated into the physical robot.

## 6. LIMITATIONS AND FUTURE DEVELOPMENT

Although the developed joint-level hardware control bridge provides a strong foundation for controlling the Dynamixel-based hexapod robot, several limitations must be acknowledged. These limitations are mainly related to hardware-specific calibration, long-duration movement validation, actuator condition, incomplete closed-loop control, and the current stage of inverse kinematics, ROS, and reinforcement learning integration.

First, although the updated calibration method is more repeatable than directly copying an imbalanced physical standing pose, it is still hardware-specific. The clean code-perfect reference values, measured physical-flat readings, and final calibrated `READY_POSE` help make the calibration process clearer, but another hexapod may still require its own calibration values because of servo horn placement, bracket tolerance, actuator orientation, mechanical load, and individual joint offsets. Therefore, the current calibration workflow improves repeatability, but it is not yet a fully automatic or universal calibration system.

Second, although the Raspberry Pi has now been integrated into the hexapod as the onboard Python control server and is powered through the robot's onboard battery system using a regulated step-down converter, the deployment still requires further long-duration validation. The current onboard setup confirms that the robot can receive wireless commands through the web interface or controller-client script, route them through `IKControl.py`, and operate without a separate Raspberry Pi power source. However, future testing should continue to evaluate startup reliability, wireless connection stability, cable management, vibration resistance, power consistency, and Raspberry Pi undervoltage behaviour during longer movement sessions.

Third, although the robot has achieved physically tested movement behaviours such as forward movement, backward movement, strafing, turning, body-height adjustment, push-up motion, and simultaneous or semi-overlapping gait-style movement, the locomotion system is still not a complete autonomous walking controller. The current movement behaviours are manually tuned and tested under controlled conditions. Future work should improve long-duration gait consistency, terrain adaptation, closed-loop balance correction, and validation under different floor surfaces, battery levels, and actuator load conditions.

Fourth, the joint-limit data may vary between individual hexapod builds. Some individual joints have been tested and observed, but a full verified minimum and maximum movement range for every hip, femur, and tibia joint is still required. This is important because the physical limits of the assembled robot may be different from the theoretical range of the Dynamixel actuator. Future testing should record safe limits for all 18 joints and store them inside the safety layer so that unsafe commands can be blocked automatically.

Fifth, some actuator behaviours require further investigation. During testing, certain joints did not retract or extend as expected, even when the motors appeared to be functional under other tools or modes. This may be caused by mechanical restrictions, horn alignment, torque settings, motor configuration, cable strain, actuator wear, or previous electrical damage. Before the robot is used for longer full-body movement sessions, each abnormal joint should be tested individually to determine whether the issue is mechanical, electrical, configuration-related, or software-related.

Sixth, the current system does not yet include a complete closed-loop locomotion controller. The robot can read feedback values such as present position, voltage, temperature, load, moving state, and warning condition, but these readings are not yet fully used to automatically adjust gait timing, body posture, foot placement, or balance during movement. Future development should use actuator feedback and additional sensors to support safer closed-loop movement correction.

Seventh, the IK Motion, IK Bézier Motion, and Body IK Posture modes are implemented as advanced movement features, but they still require further calibration and validation before they can fully replace the physically tuned fixed-gait baseline. Mathematical inverse kinematics can calculate joint targets, but the physical robot still depends on accurate calibration, joint limits, motor strength, load distribution, and real foot-ground contact.

Finally, ROS and reinforcement learning integration remain future development goals rather than completed systems. The current project provides the low-level bridge needed for these future systems by supporting motor mapping, ready-pose calibration, command routing, diagnostic monitoring, web control, controller-client communication, and onboard Raspberry Pi operation. However, future ROS nodes or reinforcement learning policies should only control the robot through this safety-limited bridge instead of directly sending raw actuator commands.

## ***6.1 Future Work***

Future development can be organised into several stages. The first stage should focus on improving the calibration workflow further. The current project separates clean code-perfect reference values, measured physical-flat readings, and the final calibrated READY\_POSE. Future work could expand this into a more automated calibration tool that guides the user through flat-pose alignment, records present motor positions, calculates mechanical offsets, and stores a per-robot calibration profile. This would make the control code easier to reuse on other similar hexapod robots without manually editing every calibration value.

The second stage should focus on strengthening hardware reliability. This includes improving wiring, confirming motor IDs, checking actuator condition, improving power stability, improving cable management, and recording safe joint limits for all 18 actuators. A more reliable hardware layer is required before the robot can be tested for longer locomotion sessions or connected to higher-level autonomous control systems.

The third stage should improve the existing movement behaviours. The current controller already supports physically tested movements such as forward movement, backward movement, strafing, turning, body-height adjustment, push-up motion, and simultaneous or semi-overlapping gait-style movement. Future work should test these behaviours for longer durations, compare performance under different battery levels, observe temperature and load changes over time, and improve gait consistency under different floor surfaces.

The fourth stage should refine IKControl.py through physical calibration. Future testing should measure the actual foot clearance produced by commanded IK lift values, tune the Bézier motion profile for smoother but efficient walking, and measure the body centre to each hip joint for more accurate Body IK. This would improve the relationship between mathematical foot targets and real physical robot movement.

The fifth stage should validate onboard Raspberry Pi powered operation during longer mobile testing sessions. In the current setup, the Raspberry Pi operates as the onboard Python server, receives wireless commands from a laptop, web browser, or controller-client script, communicates with the Dynamixel bus through the available USB, U2D2, or serial TTL interface, and receives power from the robot battery system through a regulated 5 V step-down converter. Future work should focus on improving startup automation, wireless reliability, physical mounting stability, battery runtime measurement, and longer untethered testing sessions under repeated movement conditions.

The sixth stage should improve the web-based diagnostic dashboard. The web interface could display live actuator position, voltage, temperature, load, warning status, current movement mode, battery condition, and calibration status. This would make hardware testing easier because unsafe conditions or abnormal actuator behaviour could be detected more quickly during operation.

The seventh stage should add ROS-compatible communication. The current Python bridge already performs middleware-like functions, including command routing, controller input handling, actuator feedback reading, movement execution, calibration handling, and diagnostic monitoring. Future ROS integration should build on top of this tested bridge so that ROS nodes can send high-level commands without bypassing the safety, calibration, and diagnostic layers.

The final stage should explore reinforcement learning, computer vision, terrain adaptation, and safer simulation-to-real transfer. Reinforcement learning should first be trained and tested in simulation. The output should then be converted into bounded joint commands and passed through the hardware bridge before reaching the physical motors. This approach would reduce the risk of unsafe exploration while still allowing the project to move toward autonomous hexapod locomotion.

## 7. CONCLUSION

This study presented the design, implementation, and experimental development of a Python-based joint-level control bridge for a Dynamixel-based hexapod robot. The original robot platform relied on the CM-530 controller and RoboPlus Motion workflow, where movement is commonly executed through predefined motion pages. Although this approach is useful for basic demonstrations, it limits flexibility for research development involving direct joint-level control, web-based teleoperation, controller input mapping, onboard Raspberry Pi integration, inverse kinematics, reinforcement learning, and custom robotics middleware.

A major outcome of the project is the development of a practical calibration and diagnostic workflow. The updated calibration method separates the clean code-perfect reference pose, the measured physical-flat calibration readings, and the final calibrated `READY_POSE`. This improved the repeatability of the calibration process because the robot no longer depends only on a previously captured imbalanced physical standing pose. The method also makes it easier to record hardware-specific offsets caused by servo horn placement, bracket tolerance, actuator orientation, and mechanical outliers.

The results show that real hexapod control requires more than simply sending target position values to motors. During testing, actuator behaviour was affected by voltage level, battery condition, temperature, load distribution, mechanical friction, wiring reliability, actuator condition, leg balance, and ground contact. The Dynamixel load value was useful as an approximate indicator of stress or resistance, but it could not be treated as an exact torque measurement. Therefore, safe locomotion requires a combination of software limits, actuator feedback monitoring, physical observation, and repeated hardware testing.

The developed system successfully transformed the hexapod into a more programmable experimental platform. The main current controller, `IKControl.py`, provides fixed gait fallback, IK Motion, IK Bézier Motion, movement mode switching, motor mapping, ready-pose execution, terminal control, web-based control, controller-client communication, preliminary Body IK Posture testing, walking, strafing, turning, body-height adjustment, push-up motion, and actuator health checking. The preserved `SControl.py` controller remains useful as a stable hardcoded joint-space baseline for fallback and comparison.

The project also achieved onboard Raspberry Pi operation. In the final hardware integration, the Raspberry Pi acts as the onboard Python control server and receives wireless commands from the web interface or controller-client script. The Raspberry Pi is powered from the robot's onboard battery system through a regulated 5 V step-down converter, allowing the control server to operate without a separate external Raspberry Pi power source during demonstration.

Overall, the completed system provides a research-ready foundation for future development. The contribution of this work is not only the walking movement itself, but also the creation of a safer and more structured hardware-control bridge. This bridge connects high-level commands to low-level actuator execution through motor mapping, calibration values, command routing, direction correction, safety logic, and diagnostic feedback. Future systems such as ROS-compatible nodes, inverse kinematics refinement, computer vision, reinforcement learning, and simulation-to-real transfer can build on this foundation without directly bypassing the calibrated actuator-control layer.

## REFERENCES

- Lizarraga, J. A., Garnica, J. A., Ruiz-Leon, J., Munoz-Gomez, G., & Alanis, A. Y. (2024). Advances in the kinematics of hexapod robots: An innovative approach to inverse kinematics and omnidirectional movement. *Applied Sciences*, 14(18), 8171. <https://doi.org/10.3390/app14188171>
- Ma, J., Qiu, S., Yang, J., Zhang, H., & Zhang, X. (2022). Design, analysis and experiments of hexapod robot with six-link legs for high dynamic locomotion. *Micromachines*, 13(9), 1404. <https://doi.org/10.3390/mi13091404>
- Ma'arif, S., Susanti, D. A., & Malik, M. (2022). Implementation of inverse kinematics method for self-moving on hexapod robot. *Proceeding of International Conference on Science, Engineering & Technology*.
- Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., Wheeler, R., & Ng, A. Y. (2009). ROS: An open-source robot operating system. *ICRA Workshop on Open Source Software*.
- ROBOTIS. (n.d.). DYNAMIXEL AX-12A e-Manual. *ROBOTIS e-Manual*. <https://emanual.robotis.com/docs/en/dxl/ax/ax-12a/>
- ROBOTIS. (n.d.). DYNAMIXEL SDK. *ROBOTIS e-Manual*. [https://emanual.robotis.com/docs/en/software/dynamixel/dynamixel\\_sdk/](https://emanual.robotis.com/docs/en/software/dynamixel/dynamixel_sdk/)
- ROBOTIS. (n.d.). DYNAMIXEL FAQ. *ROBOTIS e-Manual*. [https://emanual.robotis.com/docs/en/popup/faq\\_dynamixel/](https://emanual.robotis.com/docs/en/popup/faq_dynamixel/)

## APPENDIX A: System Architecture and Communication Flow

This appendix provides supporting diagrams for the final system architecture, hardware power and communication flow, software command flow, controller-based command flow, and web-based command flow used in the developed hexapod joint-level control bridge. These diagrams support the methodology and results sections by showing how external commands are routed through the Python control server before being converted into Dynamixel actuator commands.

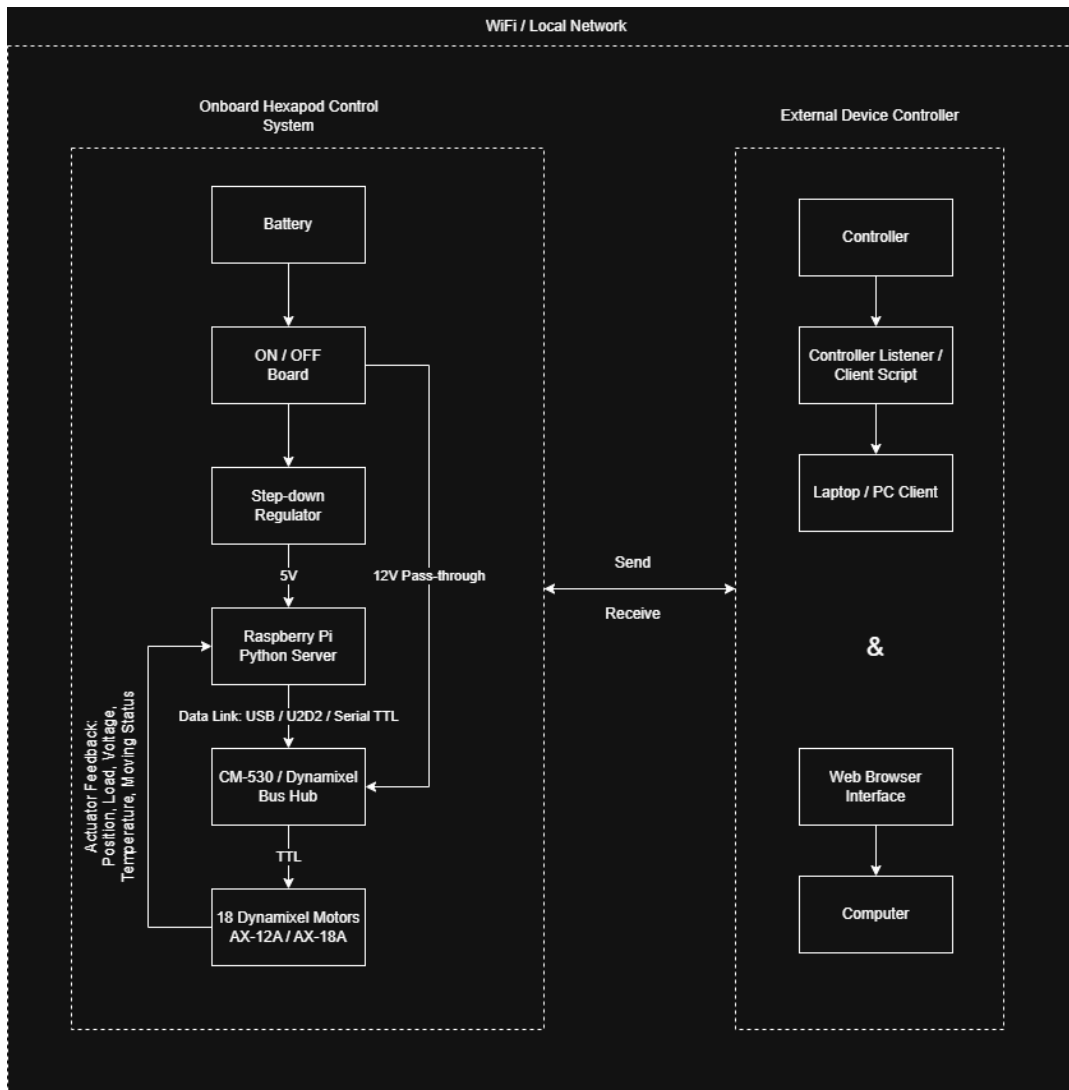
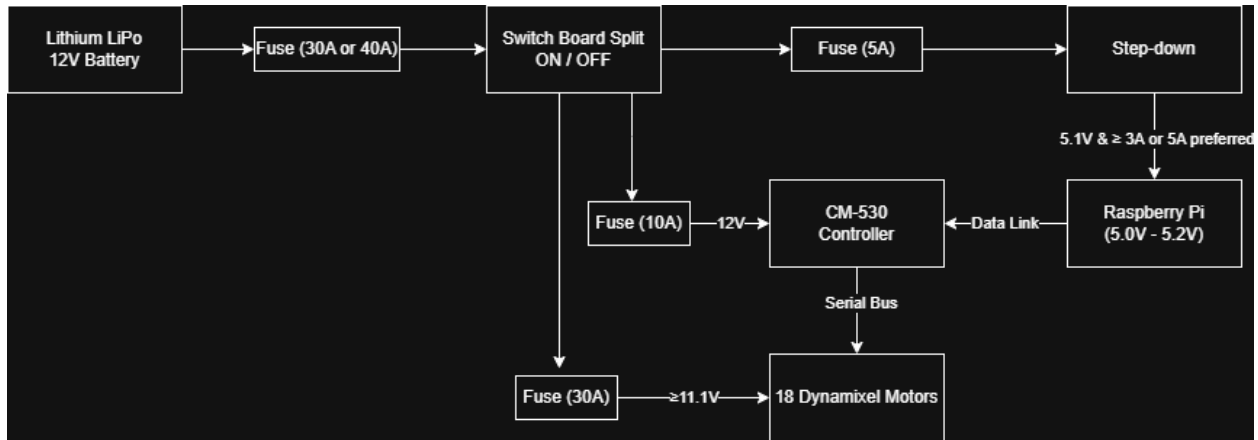
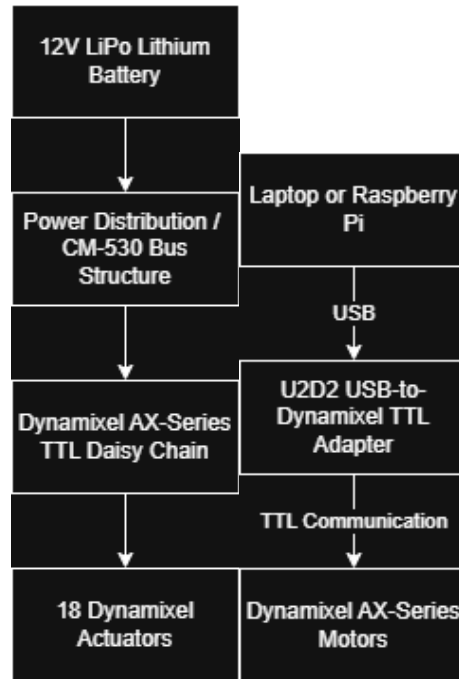


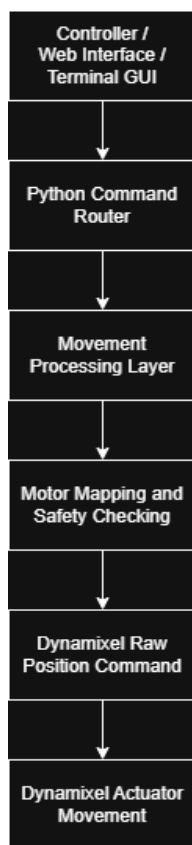
Figure A.1: Final System Architecture Map Used for the Hexapod Control Bridge



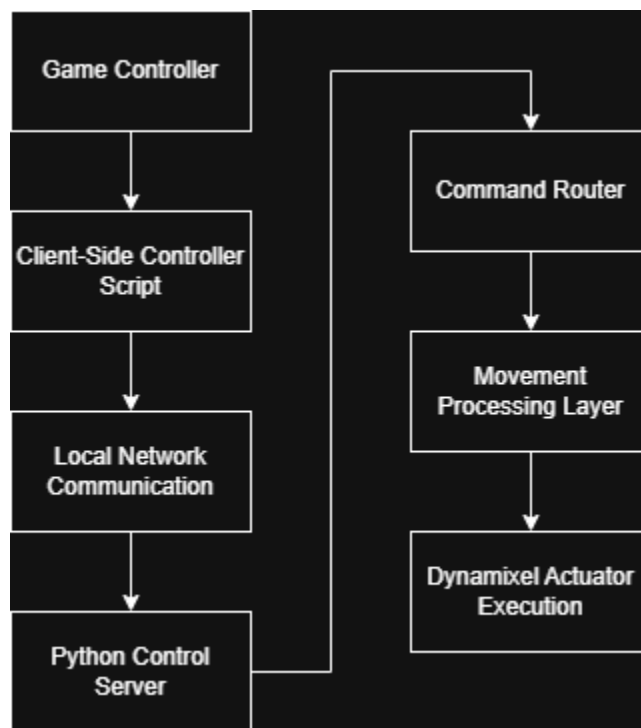
**Figure A.2: Schematic Diagram of the Onboard Battery Power Distribution for Raspberry Pi and Dynamixel Actuator System**



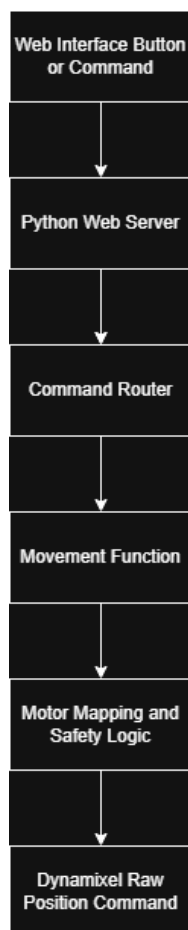
**Figure A.3: Hardware Power and Communication Flow Between Battery, Raspberry Pi, CM-530, and Dynamixel Actuators**



**Figure A.4: Software Command Flow from Web Interface and Controller Client to Dynamixel Actuator Execution**



**Figure A.5: Controller-Based Communication Flow from External Game Controller to Dynamixel Actuator Execution**



**Figure A.6: Web-Based Command Flow from Interface Button to Dynamixel Raw Position Command**

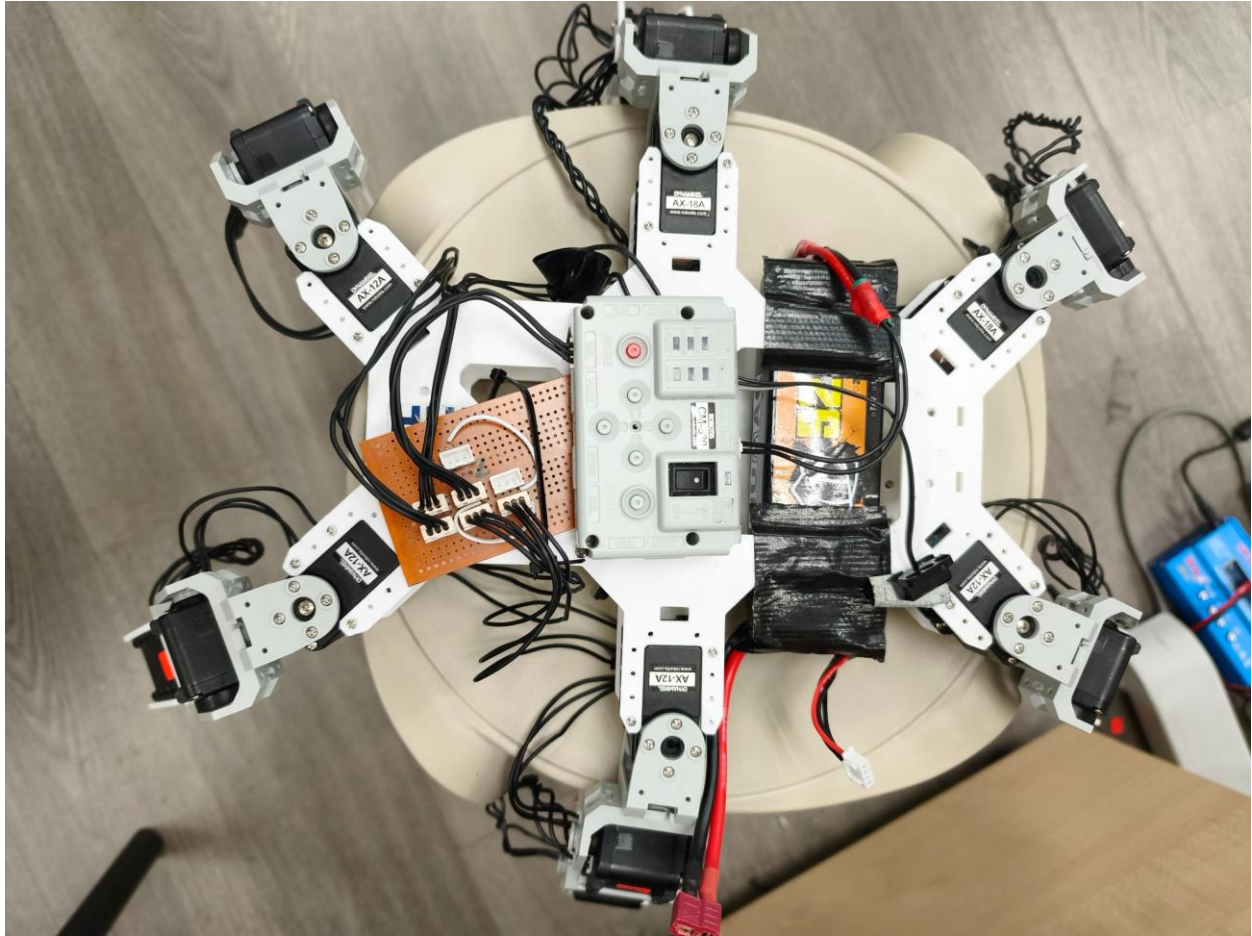
## APPENDIX B: Physical Hardware Setup



**Figure B.1: Physical Dynamixel-Based Hexapod Robot Platform Used in This Project**



**Figure B.2: Onboard Hardware Arrangement of the Hexapod Robot**



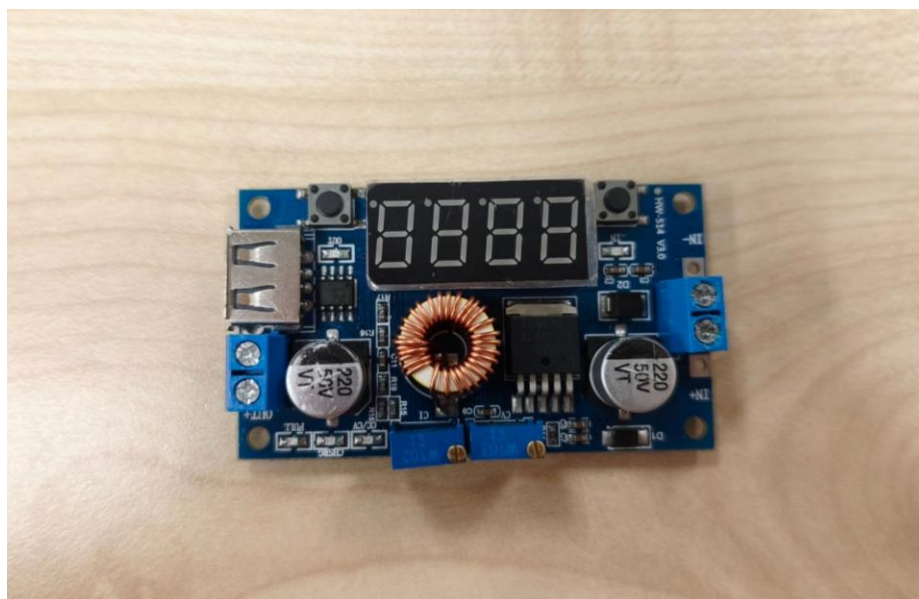
**Figure B.3: CM-530, Battery, and Custom Wiring Arrangement on the Hexapod Body**



**Figure B.4: Top View of Dynamixel AX-Series Actuator Arrangement Across the Hexapod Structure**



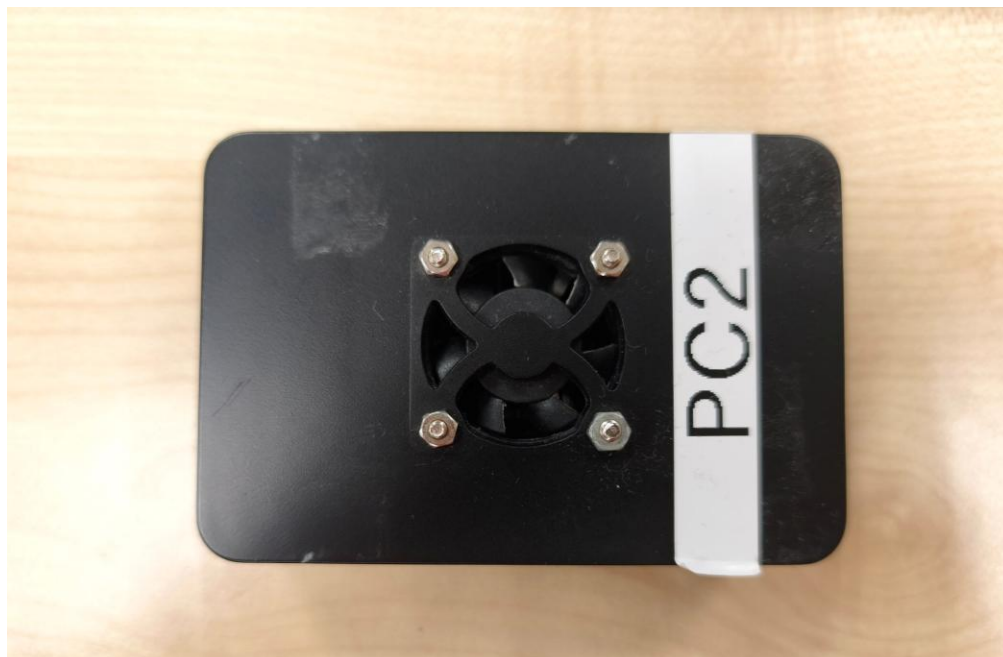
**Figure B.5: 12V LiPo Battery Used for Robot Power Supply**



**Figure B.6: Step-Down Voltage Regulator Used for Onboard Power Regulation**



**Figure B.7: USB-to-Dynamixel Communication Adapter Used for TTL Bus Access**



**Figure B.8: Laptop or Onboard Client Device Used During Hardware Testing**



**Figure B.9: External Game Controller Used for Wireless Movement Command Input**



**Figure B.10: Rockfire USB Wireless Receiver Used to Connect the Game Controller to the Client Device**

## APPENDIX C: Software Package and Controller Scripts

This appendix provides supporting evidence of the software structure used in the hexapod joint-level control bridge. The software package includes IKControl.py as the main current controller, SControl.py as the preserved hardcoded fallback controller, ControllerClient.py as the controller-client script, and supporting diagnostic, calibration, and hardware reference files. These files support command routing, web-based control, controller-client communication, actuator feedback monitoring, ready-pose calibration, movement execution, IK Motion, IK Bézier Motion, and preliminary Body IK Posture testing. The full source code is provided separately in the project software package, while this appendix shows the main software structure and key script evidence.

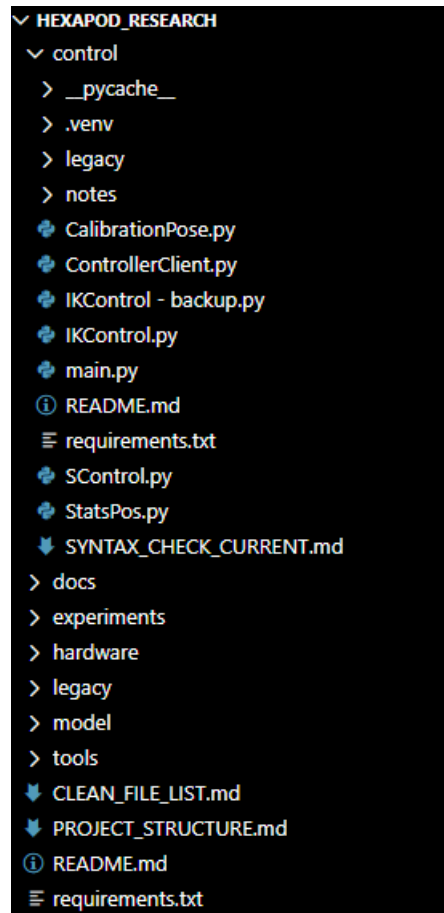


Figure C.1: Software Package Structure of the Hexapod Joint-Level Control Bridge

```

=====
SERIAL PORT DETECTION
=====
Detected serial ports:
  1) COM3          Standard Serial over Bluetooth link (COM3)
      HWID: BTHENUM\{00001101-0000-1000-8000-00805F9B34FB}_LOCALMFG&0000\8&B3D45BB&0&000000000000_0000
0000
  2) COM4          Standard Serial over Bluetooth link (COM4)
      HWID: BTHENUM\{00001101-0000-1000-8000-00805F9B34FB}_LOCALMFG&005D\8&B3D45BB&0&9C4952B36C0C_C000
0000
  3) COM5          USB Serial Port (COM5)
      HWID: USB VID:PID=0403:6001 SER=AC00H5QMA
  4) COM6          USB Serial Port (COM6)
      HWID: USB VID:PID=0403:6001 SER=A40129VXA

Choose serial port:
- Type menu number, e.g. 1
- Type exact port, e.g. COM6, /dev/ttyUSB0, /dev/ttyACM0
- Type auto or press Enter to use the first detected port
- Type rescan to scan again
Port choice [auto]: 4
Selected port: COM6

=====
CONNECTING
=====
Port: COM6
Baud: 1000000
Connected.

SControlX2 WEB UI HOLD-RELEASE MODE
Startup defaults applied: sidestrafe good, movestats off, sideflow on, speed all 25.
Startup: NO automatic movement. Press READY manually when robot is safe.
Open from laptop browser: http://<raspberrypi-ip>:8000 or http://raspberrypi.local:8000
Install web dependencies if needed: pip install fastapi uvicorn
[11:51:31] Connected to Dynamixel bus.
[11:51:31] Selected port: COM6
[11:51:31] Startup defaults applied: sidestrafe good | movestats off | sideflow on | speed all 25
[11:51:31] No automatic movement was sent. Use READY button or terminal 'r' when safe.
[11:51:31] Health is cached. Browser polling no longer reads Dynamixel motors; press Health to refres
h.
[11:51:31] Open web dashboard on port 8000. Hold-release movement is enabled.
INFO:      Started server process [50616]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)

```

Figure C.2: IKControl.py Running as the Main Hexapod Control Server

### Controller Layout

Hold button/key = move. Release = return to READY after current phase. Z/X/C control persistent stance height.

Left side movement pad      Right side face buttons

W  
Forward

△  
Health

Q  
Turn L

A  
Strafe L

D  
Strafe R

□  
Ready

⌂  
STOP

⬆  
Status

S  
Back

E  
Turn R

×  
Force R

### Stance Shortcuts

Use Z / X / C or click the buttons below. These controls are persistent until Reset Height.

Z  
Lower Stance

X  
Raise Stance

C  
Reset Height

Safe Start Check: Ready → Health

RELEASE / RETURN TO READY

### Movement Control Mode

Switch between Fixed Gait, IK Motion, IK Bézier Motion, and Body IK posture testing. These buttons only change the motion mode.

Fixed Gait

IK Motion

IK Bézier Motion

Bézier Demo Motion

Bézier ON

Bézier OFF

Body IK Posture

IK Status

Body IK Status

Health Refresh

### Quick Presets

SideStrafe Good

SideFlow ON

SideFlow OFF

Smooth Fullstep

Smooth Smoothfull

WalkLift Clear

Speed All 25

Move Stats OFF

Health Refresh

### Cached Health / State

Status  
**NOT\_READ**

Connected  
--/18

Max Temp  
-- C

Min Volt  
-- V

Max Load  
--

Motion  
idle

### Motor Status Board

Last p read: 17:27:28 · 18 motor rows

Refresh Motor Status Board (p)

OK - all motor rows normal

| ID | Joint    | Leg | Part  | Raw | DegZero | Goal | Load |
|----|----------|-----|-------|-----|---------|------|------|
| 1  | RL_hip   | RL  | hip   | 520 | +0.00   | 520  | +0   |
| 2  | FL_hip   | FL  | hip   | 520 | +0.00   | 520  | +0   |
| 3  | FR_femur | FR  | femur | 419 | +1.47   | 414  | +10  |
| 4  | FL_femur | FL  | femur | 354 | +0.88   | 351  | +96  |
| 5  | FR_tibia | FR  | tibia | 809 | -0.29   | 810  | -72  |
| 6  | FL_tibia | FL  | tibia | 808 | -0.59   | 810  | +0   |
| 7  | MR_hip   | MR  | hip   | 575 | +0.00   | 575  | +0   |
| 8  | ML_hip   | ML  | hip   | 520 | +0.00   | 520  | +0   |
| 9  | MR_femur | MR  | femur | 613 | -1.47   | 618  | -96  |
| 10 | ML_femur | ML  | femur | 417 | +0.88   | 414  | +96  |

### Main Tuning

All Speed: 55

Stance Height: 0

Lower Stance

Raise Stance

Lowest Stance

Reset Height

Highest Stance

Stance Height is persistent: movement and Ready use the selected height until Reset Height is pressed. Keyboard: Z lower, X raise, C reset.

Walk Lift Preset: clear

Walk Lift Level: 6

IK Swing / Step: 0.5

Phase Hold: 0.30

Settle: 0.14

Advanced debug / all script features

### Terminal / Debug Output

Scroll up to inspect old output. Auto-follow pauses when you scroll away from the bottom.

Auto-follow ON

```

[17:27:59] Release detected for forward; returning to READY now.
[17:27:59] Fast hold-release return to READY from forward...
[17:27:59] Fast READY recovery sent. Idle is available now.
[17:27:59] Motion thread idle. Ready for next fresh hold command.
[17:28:02] HOLD Motion START: turn_right
[17:28:04] turn_right hold cycle 1
[17:28:04] turn_right hold cycle 2
[17:28:05] turn_right hold cycle 3
[17:28:07] turn_right hold cycle 4
[17:28:09] turn_right hold cycle 5
[17:28:11] turn_right hold cycle 6
[17:28:13] turn_right hold cycle 7
[17:28:14] turn_right hold cycle 8
[17:28:15] STOP requested. Hold-release mode will return to READY after the current phase.
[17:28:16] Release detected for turn_right; returning to READY now.
[17:28:16] Fast hold-release return to READY from turn_right...
[17:28:16] Fast READY recovery sent. Idle is available now.
[17:28:16] Motion thread idle. Ready for next fresh hold command.
[17:28:21] STOP requested. Hold-release mode will return to READY after the current phase.
[17:28:22] > force r
[17:28:22] FORCE R: returning without safety check.
[17:28:24] ACTION: FAST TRIPPO LEFT RETURN TO READY
[17:28:24] [REARM] ready/recover: torque on, torque limit 3823, speed 55
[17:28:24]
[17:28:24] AFTER READY
[17:28:24]
[17:28:24] Current mode: READY REFINEDK
[17:28:24] Connected: 18/18
[17:28:24] Max temp: 38 C
[17:28:24] Min voltage: 10.7 V
[17:28:24] Max abs load: 448
[17:28:24] No reply: false
[17:28:24] Status: WARN

```

Type terminal command: r, health, speed all 25, sidestrafe @    Send

### Serial / COM Port Switch

Current shared Dynamixel bus: COM6

COM6 — USB Serial Port (COM6) [USB VID:PID=0403:6001 SER=]

optional exact port e.g. /dev/ttyUSB1 or COM8    Refresh COMs

Switch COM

COM list loaded: 4 port(s).

Web terminal commands also work: ports, port, switch com /dev/ttyUSB1, com COM6.

### Live Robot Status

MOTION MODE  
IK Motion

CURRENT COMMAND  
READY REFINEDK

STANCE HEIGHT  
0 / -7.7 F 0 / T 0

WALK LIFT  
F -25 / T 28

IK LIFT  
7 cm

BÉZIER  
OFF

SMOOTH / STEP END  
OFF / tripod

SIDEFLOW / MOVESTATS  
ON / OFF

**Figure C.3: Full Web-Based Control Interface Showing Movement Controls, Tuning Panels, Terminal Output, COM Port Switching, and Motor Status Feedback**

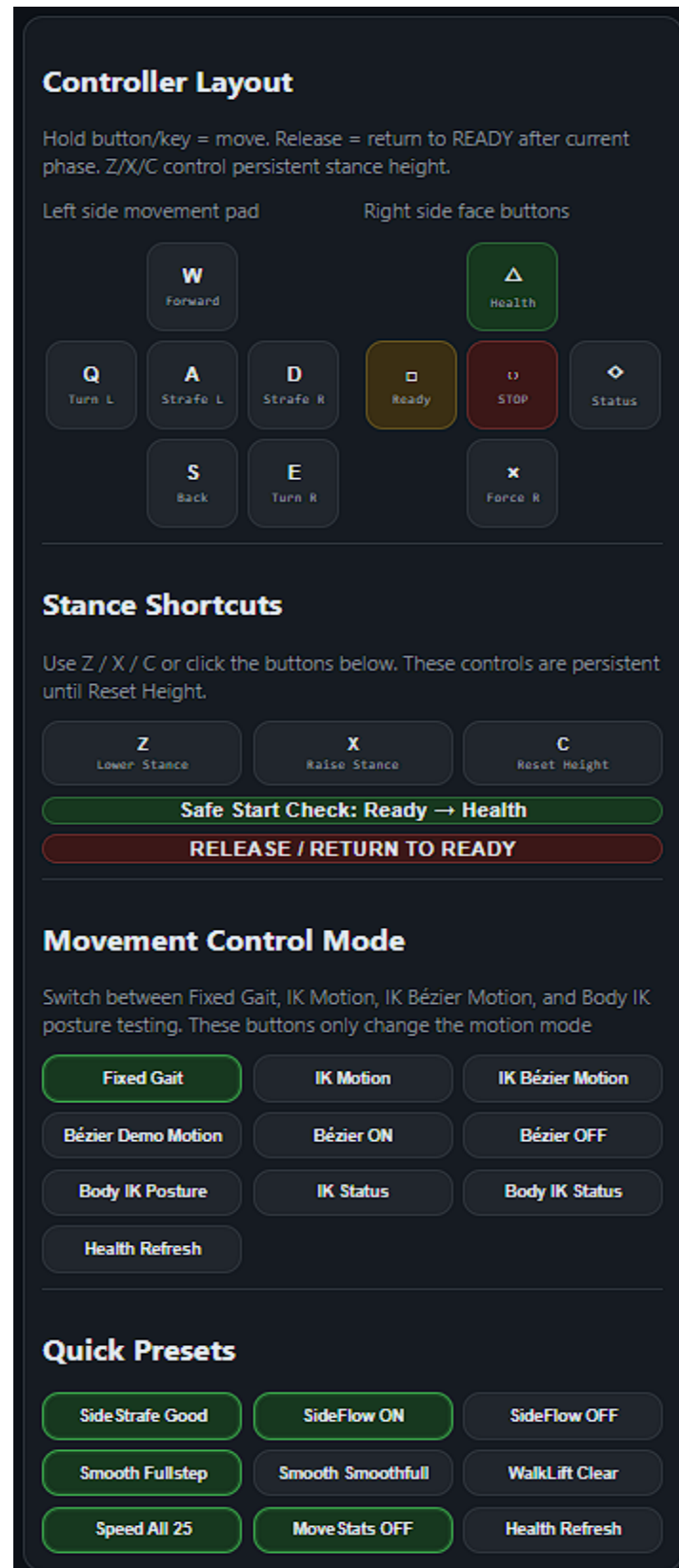


Figure C.3(a): Controller Layout Panel for Manual Hexapod Movement Commands

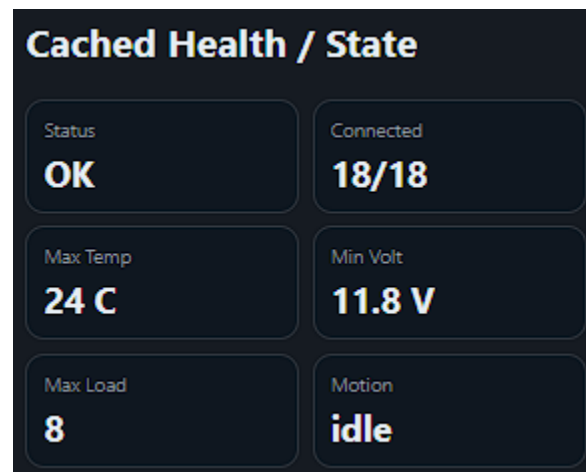


Figure C.3(b): Cached Health and Robot State Panel in the Web Interface

### Main Tuning

All Speed  25

Stance Height  0

Lower Stance Raise Stance Lowest Stance

Reset Height Highest Stance

Stance Height is persistent: movement and Ready use the selected height until Reset Height is pressed. Keyboard: Z lower, X raise, C reset.

Walk Lift Preset

Walk Lift Level  6

Phase Hold  0.30

Settle  0.14

▼ **Advanced debug / all script features**

Forward Range   Set

Strafe Range   Set

Turn Range   Set

Lift Legs   Lift

Lift Tripod A Lift Tripod B Torque Max

MoveStats ON MoveStats OFF Smooth ON

Smooth OFF Pushup 1 Pushup 4

Lift All L7 Latency FAST Latency NORMAL

Figure C.3(c): Main Tuning Panel for Movement Speed, Stance Height, and Gait Adjustment

### Terminal / Debug Output

Type terminal command: r, health, speed all 25, sidestrate g Send

### Serial / COM Port Switch

Current shared Dynamixel bus: **COM3**

**COM3 — Standard Serial over Bluetooth link (COM3) [BTENUM/**

optional exact port e.g. /dev/ttyUSB1 or COM6 Refresh COMs

Switch COM

COM list loaded: 2 port(s).

Web terminal commands also work: **ports, port, switch com**  
**/dev/ttyUSB1, com COM6.**

### Live Robot Status

|                                             |                                         |
|---------------------------------------------|-----------------------------------------|
| MOTION MODE<br><b>Fixed Gait</b>            | CURRENT COMMAND<br><b>Idle / Ready</b>  |
| STANCE HEIGHT<br><b>0 / -7..7 F 0 / T 0</b> | WALK LIFT<br><b>F -28 / T 28</b>        |
| IK LIFT<br><b>7 cm</b>                      | BÉZIER<br><b>OFF</b>                    |
| SMOOTH / STEP END<br><b>OFF / tripod</b>    | SIDEFLOW / MOVESTATS<br><b>ON / OFF</b> |

Figure C.3(d): Terminal Debug Output Panel Displaying Server, Command Feedback and Serial COM Port Switch

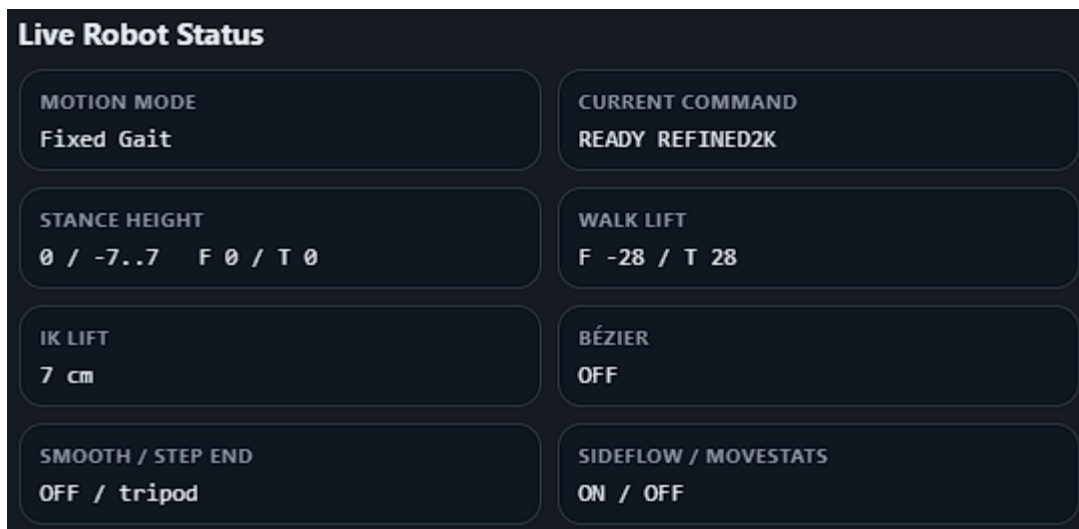


Figure C.3(e): Live Robot Status Panel in the Web-Based IKControl Interface

```

=====
Raspberry Pi web URL [http://raspberrypi.local:8000]: http://127.0.0.1:8000/

Detected controllers:
  0) Rockfire PSX/USB Bridge | buttons=12 axes=4 hats=1
Select controller index [0]: 0

=====
ROCKFIRE FULL-MODE CONTROLS V5 SMOOTH HEIGHT
=====

D-PAD
Up      = Forward
Down    = Backward
Left    = Strafe Left
Right   = Strafe Right

LEFT JOYSTICK
Axis 1 -1 = Forward
Axis 1 +1 = Backward
Axis 0 -1 = Strafe Left
Axis 0 +1 = Strafe Right

RIGHT JOYSTICK
Axis 2 -1 = Turn Left
Axis 2 +1 = Turn Right

FACE BUTTONS
Button 0 / Cross      = Reset body-height level to 0
Button 1 / Circle     = Ready at current body-height level
Button 2 / Square     = Health
Button 3 / Triangle  = Startup setup + reset body-height level

SHOULDER / TRIGGER
Button 4 / L1         = Speed -1 per click
Button 5 / R1         = Speed +1 per click
Button 6 / L2 hold   = Smooth lower body toward -7
Button 7 / R2 hold   = Smooth raise body toward +7

BEHAVIOR
Hold movement        = Move
Release movement     = Stop / return
Robot busy           = New directions ignored
Idle + neutral       = Accept next command
=====

```

Figure C.4: Controller-Client Script Running and Sending Game Controller Commands to the Server

## APPENDIX D: Motor Mapping and Ready-Pose Calibration

This appendix provides supporting evidence for the motor mapping and repeatable calibration process used by the hexapod control system. The motor mapping links each Dynamixel actuator ID to a physical joint on the robot, while the calibration evidence separates the clean code-perfect reference pose, the measured physical-flat motor readings, and the final calibrated READY\_POSE. This separation is important because each actuator may have different horn alignment, physical orientation, bracket tolerance, mechanical offset, and load behaviour. By recording the physical-flat calibration values separately from the final ready-pose dictionary, the controller can generate repeatable movement from a known hardware-specific posture instead of relying only on theoretical neutral positions or an imbalanced captured standing pose.

The repeatable flat-pose calibration method was used to align the physical servo horn positions with the software reference pose before recording the measured motor positions and final ready-pose values. This process helps another user reproduce the same starting posture when the robot is reassembled, when a servo horn is removed, or when the same control code is used on a similarly built hexapod. The appendix figures therefore support the calibration workflow described in Section 4.4.1 and the calibration results discussed in Section 5.2.



**Figure D.1: Star-Shaped Flat-Pose Overhead View Used for Horn Alignment**



Figure D.2: Oblique View of the Flat-Pose Horn Alignment Setup

```
JOINT_INFO = {
  "RL_hip": {"id": 1, "type": "hip"},
  "FL_hip": {"id": 2, "type": "hip"},
  "FR_femur": {"id": 3, "type": "femur"},
  "FL_femur": {"id": 4, "type": "femur"},
  "FR_tibia": {"id": 5, "type": "tibia"},
  "FL_tibia": {"id": 6, "type": "tibia"},
  "MR_hip": {"id": 7, "type": "hip"},
  "ML_hip": {"id": 8, "type": "hip"},
  "MR_femur": {"id": 9, "type": "femur"},
  "ML_femur": {"id": 10, "type": "femur"},
  "MR_tibia": {"id": 11, "type": "tibia"},
  "ML_tibia": {"id": 12, "type": "tibia"},
  "RR_hip": {"id": 13, "type": "hip"},
  "FR_hip": {"id": 14, "type": "hip"},
  "RR_femur": {"id": 15, "type": "femur"},
  "RL_femur": {"id": 16, "type": "femur"},
  "RR_tibia": {"id": 17, "type": "tibia"},
  "RL_tibia": {"id": 18, "type": "tibia"},
}
```

Figure D.3: Dynamixel Motor ID Mapping Used by the Hexapod Control System

```

CALIBRATION_PHYSICAL_FLAT_RAW = {
    # hips/coxa use clean calibration reference, not the accidental lay-flat spread
    1: 520,    # RL_hip
    2: 520,    # FL_hip
    7: 575,    # MR_hip mechanical outlier
    8: 520,    # ML_hip
    13: 520,   # RR_hip
    14: 520,   # FR_hip

    # femurs measured in physical flat pose
    3: 513,    # FR_femur = 505 + 8
    4: 456,    # FL_femur = 505 - 49
    9: 505,    # MR_femur = 505 + 0
    10: 529,   # ML_femur = 505 + 24
    15: 507,   # RR_femur = 505 + 2
    16: 529,   # RL_femur = 505 + 24

    # tibias measured in physical flat pose
    5: 364,    # FR_tibia = 365 - 1
    6: 364,    # FL_tibia = 365 - 1
    11: 367,   # MR_tibia = 365 + 2
    12: 364,   # ML_tibia = 365 - 1
    17: 361,   # RR_tibia = 365 - 4
    18: 357,   # RL_tibia = 365 - 8
}

```

**Figure D.4: Captured Physical-Flat Calibration Dictionary Used to Record Measured Motor Positions**

**Figure D.4** shows the physical-flat calibration dictionary captured after the flat-pose horn-alignment procedure. These values represent the measured motor positions of the assembled robot when the legs were physically aligned and pressed down evenly. The dictionary records the real hardware pose and is used to identify mechanical offsets relative to the clean code-perfect calibration reference.

```

READY_POSE = {
  # HIPS / COXA:
  1: 520,
  2: 520,
  7: 575,
  8: 520,
  13: 520,
  14: 520,

  # FEMURS:
  3: 413,
  4: 356,
  9: 405,
  10: 429,
  15: 407,
  16: 429,

  # TIBIAS:
  5: 801,
  6: 801,
  11: 804,
  12: 801,
  17: 798,
  18: 794,
}

```

**Figure D.5: Final Calibrated READY\_POSE Dictionary Used for Stable Movement Initialization**

Figure D.5 shows the final calibrated READY\_POSE dictionary used by the controller as the stable movement initialization posture. Unlike Figure D.4, which records the measured physical-flat calibration values, this dictionary represents the practical standing reference used during movement execution. The controller applies logical movement offsets from this calibrated base position when generating Dynamixel raw-position targets.

```

=====
  TERMINAL HEALTH CHECK
=====
Current mode : UNKNOWN
Connected    : 18/18
Max temp     : 31 C
Min voltage  : 12.0 V
Max abs load : 0
No reply     : False
Status       : OK

```

**Figure D.6: Motor Status Output Used to Verify Ready-Pose Calibration and Actuator Condition**

## APPENDIX E: Basic Diagnostic and Health Check Outputs

This appendix provides selected examples of diagnostic and health-check outputs used during hexapod testing. The outputs are included only as representative evidence to show that the controller can read actuator status, verify motor communication, and monitor basic hardware conditions such as voltage, temperature, load, and motor position. Full diagnostic commands, help menus, and detailed usage instructions are provided separately in the project README and software guide files.

These diagnostic outputs also support the web-based status panel described in Section 5.8, where recent motor-status information can be reviewed without relying only on scrolling terminal output.

```
=====
SERIAL PORT DETECTION
=====
Detected serial ports:
  1) COM3          Standard Serial over Bluetooth link (COM3)
      HWID: BTHENUM\{00001101-0000-1000-8000-00805F9B34FB}_LOCALMFG&0000\8&B3D45BB&
0&000000000000_00000000
  2) COM4          Standard Serial over Bluetooth link (COM4)
      HWID: BTHENUM\{00001101-0000-1000-8000-00805F9B34FB}_LOCALMFG&005D\8&B3D45BB&
0&9C4952B36C0C_C0000000
  3) COM5          USB Serial Port (COM5)
      HWID: USB VID:PID=0403:6001 SER=AC00H5QMA
  4) COM6          USB Serial Port (COM6)
      HWID: USB VID:PID=0403:6001 SER=A40129VXA

Choose serial port:
- Type menu number, e.g. 1
- Type exact port, e.g. COM6, /dev/ttyUSB0, /dev/ttyACM0
- Type auto or press Enter to use the first detected port
- Type rescan to scan again
Port choice [auto]:
```

Figure E.1: Serial Port Detection and Dynamixel Bus Connection Output

```

SControlX2 command [h help]: health

=====
  TERMINAL HEALTH CHECK
=====
Current mode : UNKNOWN
Connected    : 18/18
Max temp     : 25 C
Min voltage  : 11.8 V
Max abs load : 0
No reply     : False
Status       : OK

```

Figure E.2: Terminal Health Check Output Showing Connected Motors, Voltage, Temperature, Load, and Status

```

SControlX2 command [h help]: p

=====
  MOTOR STATUS / FULL DIAGNOSTICS
=====

```

| ID | Joint    | Leg Part | Raw | DegZero | Zero | Goal | Load | Volt | Temp | Warnings |
|----|----------|----------|-----|---------|------|------|------|------|------|----------|
| 1  | RL_hip   | RL hip   | 385 | -21.99  | 460  | 460  | +0   | 12.0 | 29   | OK       |
| 2  | FL_hip   | FL hip   | 749 | +0.59   | 747  | 747  | +0   | 12.1 | 31   | OK       |
| 3  | FR_femur | FR femur | 726 | +92.38  | 411  | 411  | +0   | 12.0 | 30   | OK       |
| 4  | FL_femur | FL femur | 650 | +83.28  | 366  | 366  | +0   | 12.0 | 31   | OK       |
| 5  | FR_tibia | FR tibia | 454 | -100.88 | 798  | 798  | +0   | 12.3 | 30   | OK       |
| 6  | FL_tibia | FL tibia | 433 | -106.45 | 796  | 796  | +0   | 12.0 | 29   | OK       |
| 7  | MR_hip   | MR hip   | 539 | -20.23  | 608  | 608  | +0   | 12.0 | 29   | OK       |
| 8  | ML_hip   | ML hip   | 847 | +27.57  | 753  | 753  | +0   | 11.8 | 31   | OK       |
| 9  | MR_femur | MR femur | 270 | -104.69 | 627  | 627  | +0   | 12.1 | 30   | OK       |
| 10 | ML_femur | ML femur | 805 | +107.92 | 437  | 437  | +0   | 12.0 | 29   | OK       |
| 11 | MR_tibia | MR tibia | 421 | +60.12  | 216  | 216  | +0   | 12.0 | 29   | OK       |
| 12 | ML_tibia | ML tibia | 421 | -107.33 | 787  | 787  | +0   | 11.8 | 30   | OK       |
| 13 | RR_hip   | RR hip   | 486 | -26.98  | 578  | 578  | +0   | 12.1 | 29   | OK       |
| 14 | FR_hip   | FR hip   | 654 | +23.17  | 575  | 575  | +0   | 12.0 | 29   | OK       |
| 15 | RR_femur | RR femur | 252 | -114.08 | 641  | 641  | +8   | 12.3 | 29   | OK       |
| 16 | RL_femur | RL femur | 752 | +99.71  | 412  | 412  | +0   | 12.0 | 29   | OK       |
| 17 | RR_tibia | RR tibia | 584 | +115.84 | 189  | 189  | +0   | 12.1 | 31   | OK       |
| 18 | RL_tibia | RL tibia | 466 | -102.93 | 817  | 817  | +0   | 11.8 | 31   | OK       |

Figure E.3: Full Motor Diagnostic Output Showing Raw Position, Goal, Load, Voltage, Temperature, and Warning State

## APPENDIX F: Movement Testing Evidence

This appendix provides visual evidence of the physical movement testing performed on the Dynamixel-based hexapod robot. The images show the robot in tested postures and movement-related conditions, including ready-pose standing, body posture, actuator arrangement during movement, and the physical testing setup. These images support the results section by showing that the developed control bridge was tested on real hardware rather than only in simulation or software.

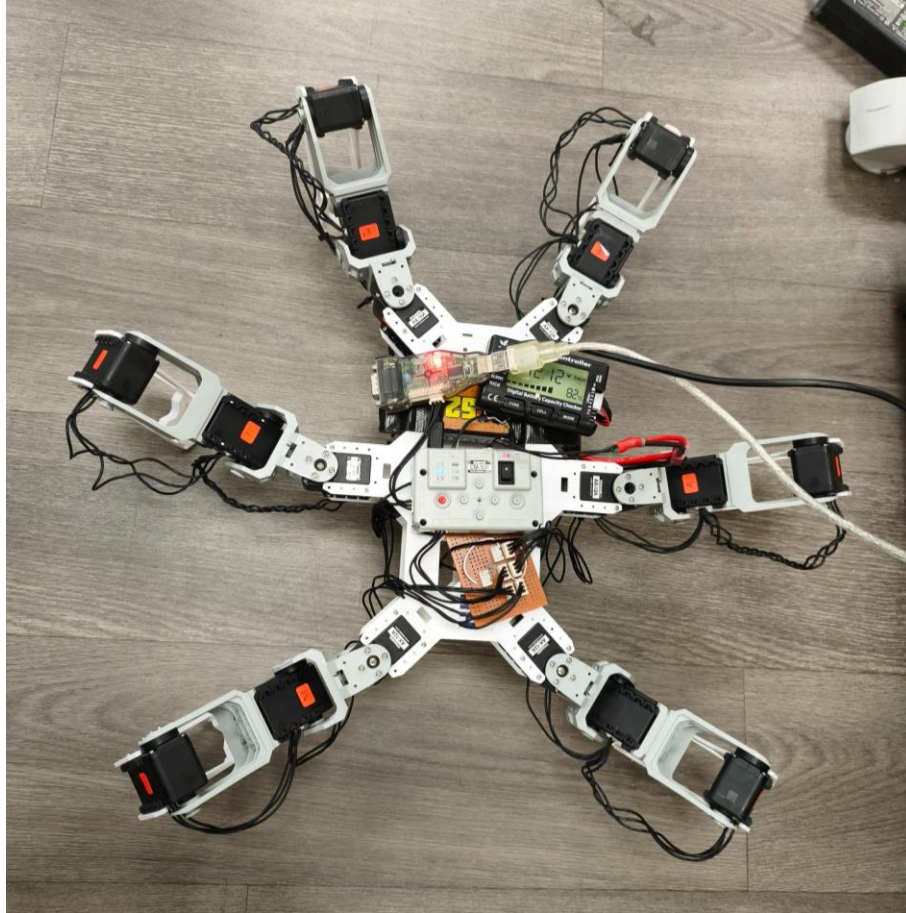
The ready-pose and gait images should be interpreted as physical movement evidence after calibration, rather than as universal joint positions for every hexapod build.



(a) Rear Top Angled View Ready Pose

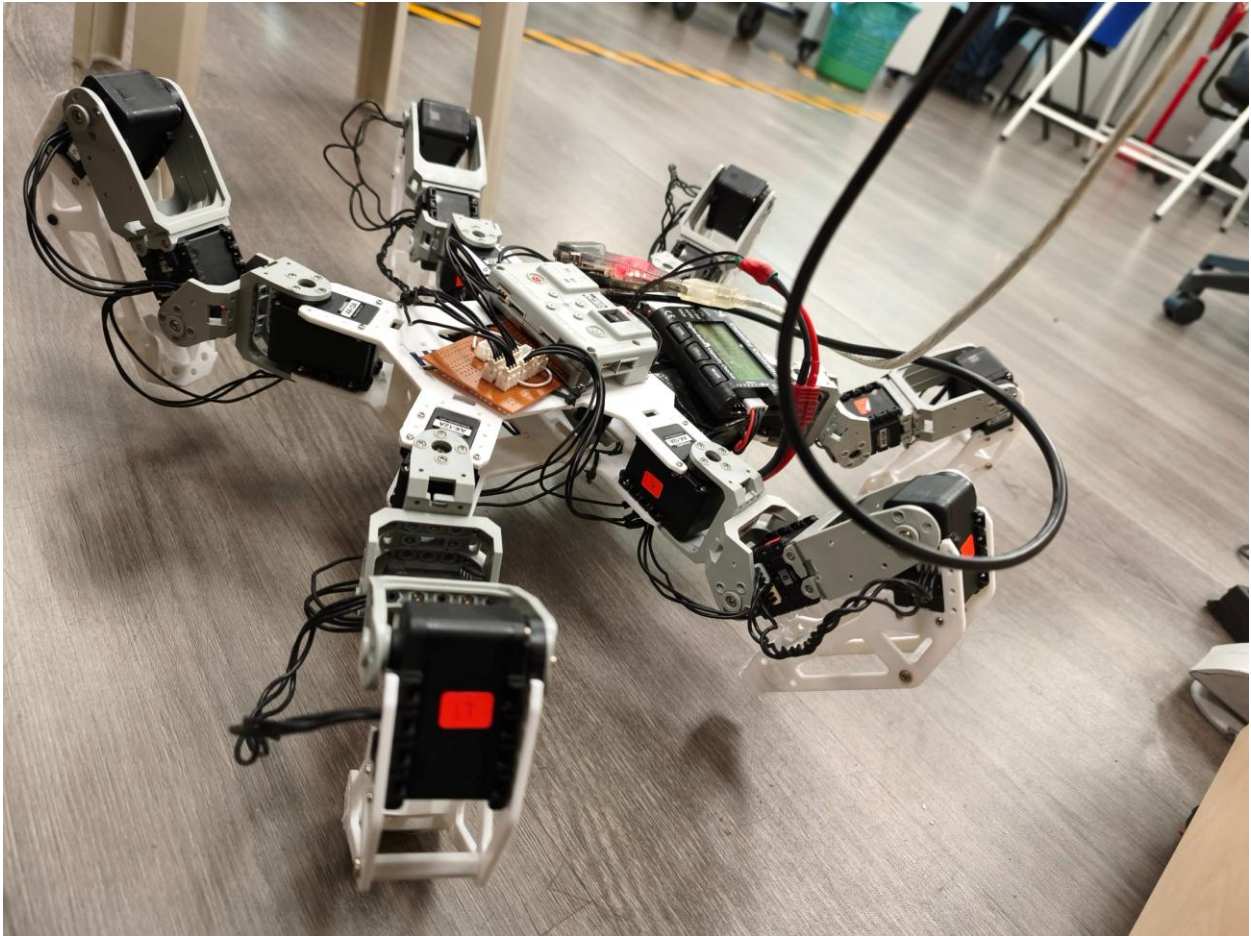


**(b) Rear Left Side View Ready Pose**



(c) Top View Ready Pose

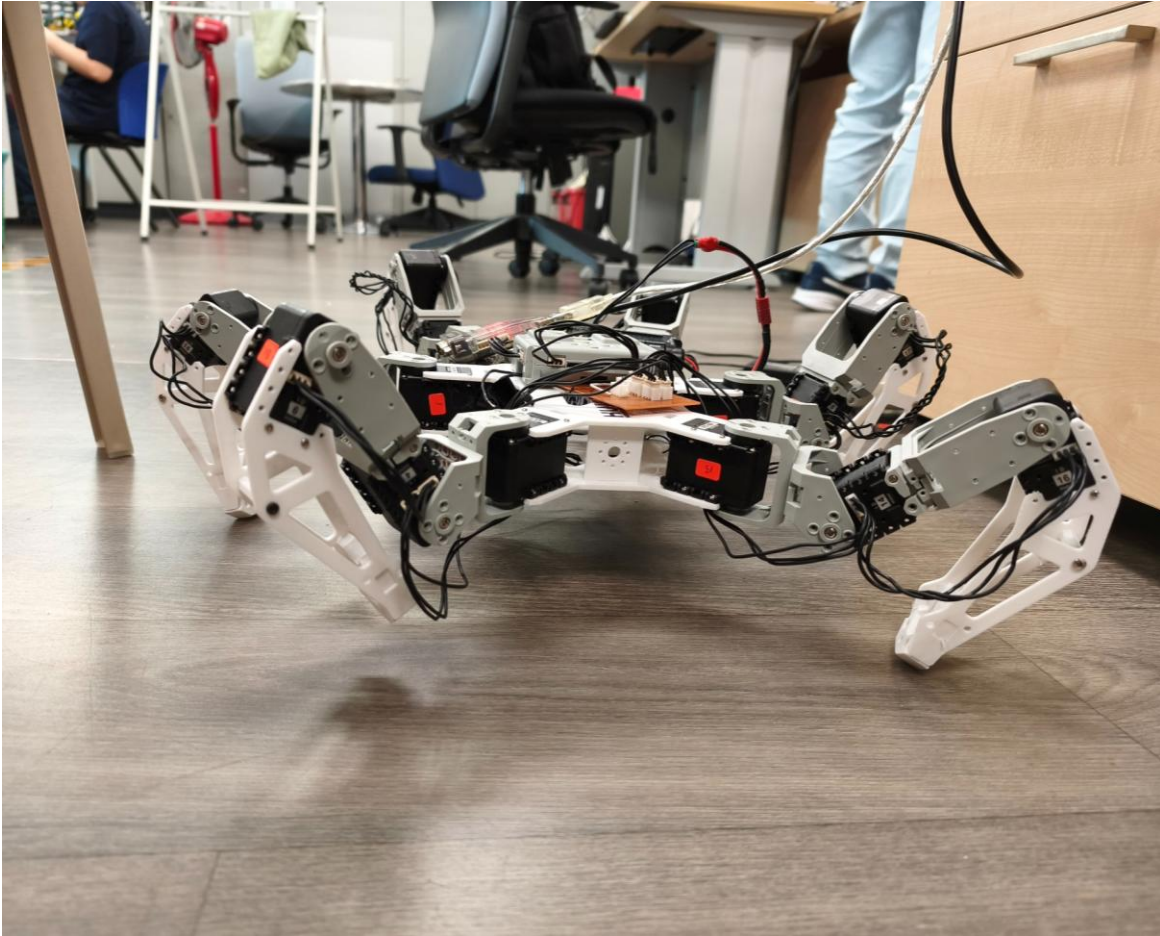
**Figure F.1: Multi-View Ready-Pose Test Showing the Hexapod in the Calibrated Standing Position**



**(a) Rear Right Top View Gait 1**

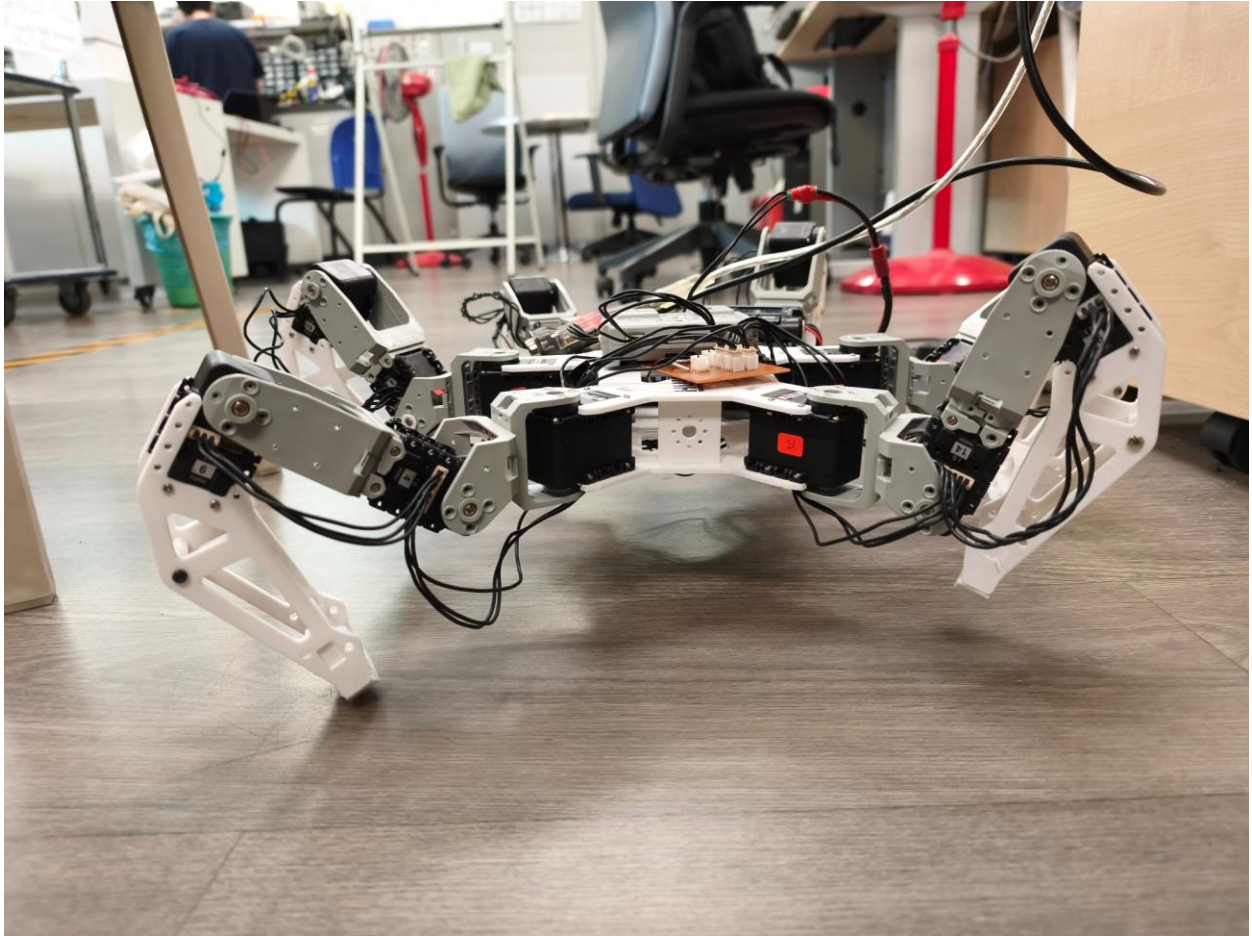


**(b) Rear Left Angled Top View Gait 1**



**(c) Rear View Gait 1**

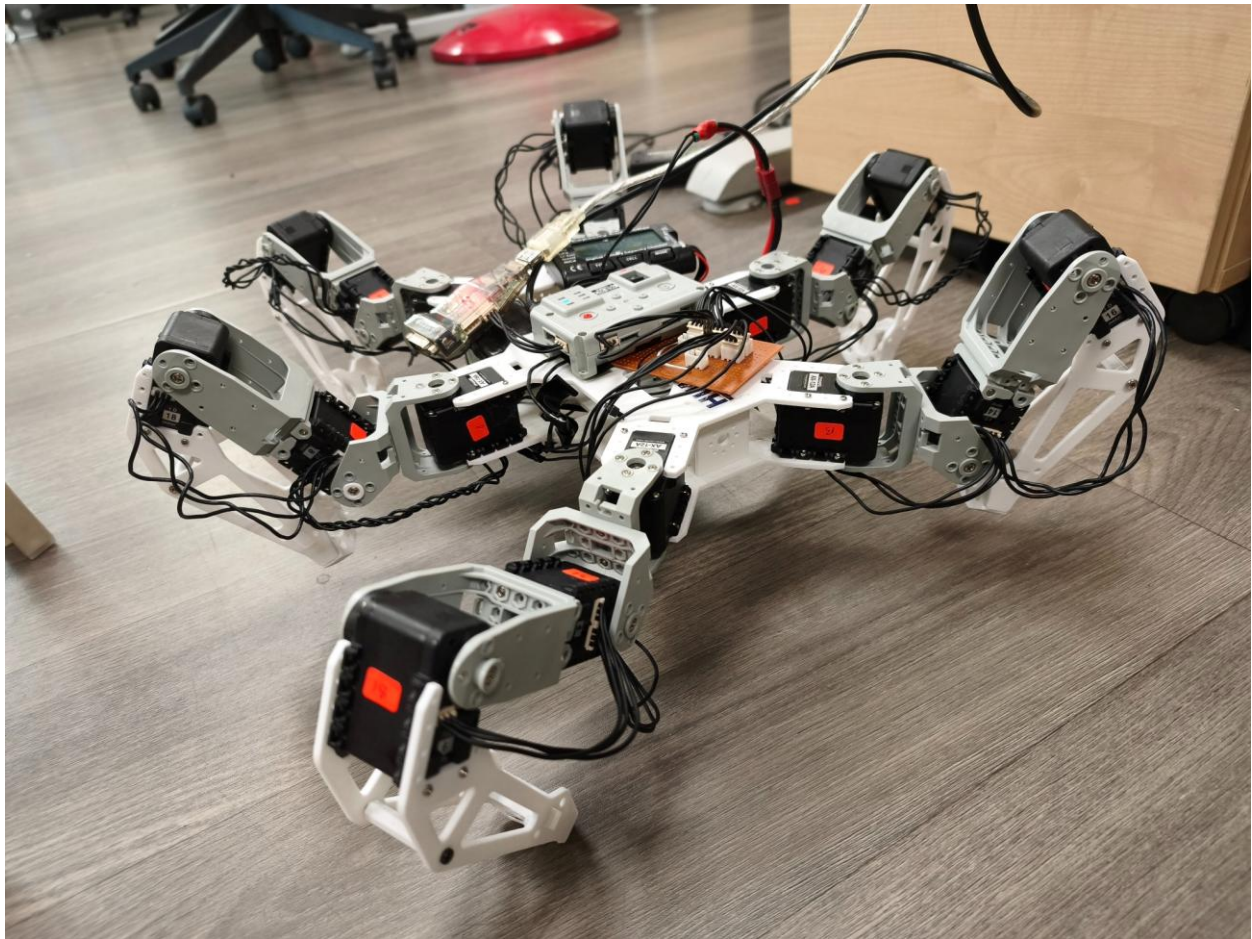
**Figure F.2: Tripod Gait Posture A with the FL, MR, and RL leg group lifted for movement validation, shown from multiple viewing angles**



**(a) Rear View Gait 2**



**(b) Rear Right View Gait 2**



(c) Rear Left Angled View Gait 2

**Figure F.3: Tripod Gait Posture B (FR–ML–RR Lifted) Shown from Multiple Viewing Angles**